
Plasticity Activation via Polar Operator: A Plug-in Method for Balancing Stability and Plasticity

Guodong Zheng¹ Enneng Yang² Xiaoyan Wang³ Yihan Chen⁴ Feihong He² Quan Zheng¹ Peng Wang²
Li Shen²

Abstract

Continual learning (CL) seeks models that acquire new knowledge while avoiding catastrophic forgetting. However, many methods that mitigate forgetting constrain parameter updates and thereby reduce model plasticity. We revisit the singular value spectrum of gradients in representative CL methods and show that they commonly exhibit singular value collapse, where only a small subset of gradient directions drive parameter updates. Motivated by this observation, we propose **Plasticity Activation via Polar Operator (PAPO)**, a plug-in that preserves the dominant directions that mitigate forgetting while activating previously suppressed directions to enhance plasticity. Concretely, PAPO modifies the gradient $\mathbf{G}$ as $\mathbf{G} \leftarrow \mathbf{G} + \lambda \cdot \text{polar}(\mathbf{G})$, which uniformly increases near-zero singular values without changing the singular vectors. To avoid the cost of explicit singular value decomposition, we approximate the polar factor using the iteration-dependent Polar Express scheme, which relies only on matrix multiplications and additions. In our empirical evaluation on both vision and language benchmarks, incorporating PAPO yields consistent improvements. In particular, on Mini-ImageNet, integrating PAPO into ER, MAS, GPM and TRGP produces substantial accuracy gains of 9.01%, 4.76%, 8.90% and 9.19%, respectively.

1. Introduction

Continual learning (CL) aims to enable models to acquire new knowledge as tasks arrive while retaining previously acquired knowledge. Prior work has focused primarily on mitigating catastrophic forgetting (McCloskey & Cohen, 1989), often by constraining parameter updates. However, overly restricting updates to prevent forgetting can impair the model’s ability to learn new tasks. This tension, commonly referred to as the stability-plasticity dilemma (Grossberg, 2013; Wang et al., 2024b), captures the inherent difficulty of preserving past knowledge while accommodating new information.

Recent work has sought to balance the stability–plasticity trade-off using diverse strategies. Examples include auxiliary-network approaches that separate stability and plasticity roles (Kim et al., 2023; Lu et al., 2025a), methods tailored to particular CL algorithms such as null-space approaches (Kong et al., 2022) and model merging (Yang et al., 2026), and methods designed for specific CL paradigms such as continual meta-learning (Chen et al., 2023) and streaming learning (Elsayed & Mahmood, 2024). Many of these approaches either require additional network components or are tailored to specific algorithms or learning scenarios, and therefore do not provide a broadly applicable and efficient solution to the stability–plasticity dilemma.

To develop a general method for enhancing plasticity in CL approaches that mitigate forgetting, we examine the gradient singular value spectra of three representative methods: Experience Replay (ER) (Chaudhry et al., 2019b), Memory Aware Synapses (MAS) (Aljundi et al., 2018), and Gradient Projection Memory (GPM) (Saha et al., 2021). By inspecting these spectra, we identify a common pattern, which we call *singular value collapse*: a small set of dominant singular values is followed by a long tail of near-zero singular values. Under singular value collapse, only a small subset of gradient directions drives parameter updates, thereby constraining model plasticity. Specifically, the spectrum of GPM exhibits a cliff that separates a few relatively large singular values from a bulk of near-zero values, and this cliff shifts leftward as tasks accumulate. We also provide a theoretical analysis showing that this phenomenon naturally

¹Huazhong University of Science and Technology, Wuhan, China ²Shenzhen Campus of Sun Yat-sen University, China ³Information Technology Service Center of People’s Court, Beijing, China ⁴University of Science and Technology of China, Hefei, China. Correspondence to: Li Shen <mathshenli@gmail.com>, Peng Wang <wangp389@mail.sysu.edu.cn>, Xiaoyan Wang <428163395@139.com>.

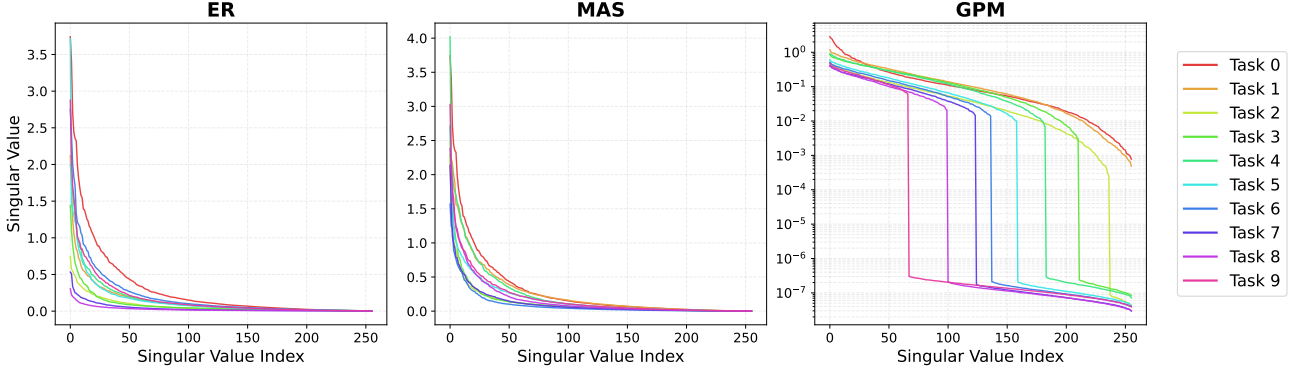


Figure 1. Gradient singular value spectra of a convolutional layer of AlexNet on CIFAR-100 for ER, MAS, and GPM across tasks.

arises in orthogonal-projection-based methods.

Motivated by the observed singular value collapse, we propose Plasticity Activation via Polar Operator (PAPO), a plug-in module that activates previously suppressed singular directions to enhance plasticity in CL methods. Specifically, PAPO modifies the gradient as $\mathbf{G} \leftarrow \mathbf{G} + \lambda \cdot \text{polar}(\mathbf{G})$, where $\text{polar}(\cdot)$ denotes the polar operator and $\lambda > 0$ is a balance coefficient. Applying the polar operator preserves the singular vectors while shifting every singular value σ_i to $\sigma_i + \lambda$. As a result, the relative ordering of singular values is maintained and the original dominant update directions are retained, while previously near-zero singular values are uniformly increased and the corresponding directions are activated. To avoid an explicit singular value decomposition (SVD), we approximate $\text{polar}(\cdot)$ with Polar Express (Am- sel et al., 2025), an iterative scheme that uses only matrix multiplications and additions and is therefore efficient on modern GPUs.

Empirically, we conduct experiments on both vision and language tasks. For vision tasks, we integrate PAPO into ER, MAS, GPM and Trust Region Gradient Projection (TRGP) (Lin et al., 2022) and evaluate the resulting methods on four standard benchmarks: PMNIST, CIFAR-100, MiniImageNet and the 5-Datasets sequence. Averaged across these benchmarks, PAPO improves the accuracy of ER, MAS, GPM and TRGP by 3.52%, 3.18%, 2.80% and 2.91%, respectively. On MiniImageNet in particular, PAPO yields substantial accuracy gains of 9.01%, 4.76%, 8.90% and 9.19%, while also improving backward transfer (BWT) by 0.03, 0.04, 0.04 and 0.01, respectively. For language tasks, we apply PAPO to InfLoRA (Liang & Li, 2024) and GainLoRA (Liang et al., 2026) and evaluate on the SuperNI benchmark (Wang et al., 2022b). PAPO improves the accuracy of InfLoRA and GainLoRA by 3.37% and 1.24%, respectively, with corresponding reductions in forgetting of 3.22% and 1.39%. These results demonstrate the practical benefits of PAPO as a plug-in that can be incorporated into methods that mitigate forgetting in CL. Our code is available

at <https://github.com/Zhenggd943/PAPO>.

Our main contributions are summarized as follows:

- We revisit the singular value spectrum of gradients in ER, MAS and GPM and identify a phenomenon we term *singular value collapse*. We also provide a theoretical analysis that explains why a spectral “cliff” arises in orthogonal-projection-based methods.
- We propose a plug-in, Plasticity Activation via Polar Operator (PAPO), which activates suppressed gradient directions in the optimization dynamics while preserving the original mitigating-forgetting directions. PAPO is broadly applicable and does not require changes to model architecture or loss functions.
- We validate PAPO through extensive experiments on both vision and language benchmarks, verifying that it consistently improves average performance across these tasks.

Related Work. Due to space constraints, a comprehensive literature review is provided in Appendix A.

2. Preliminary

In this section, we introduce the notation used throughout the paper and provide an overview of the SVD of matrices.

Continual Learning. We consider the standard CL setting with T sequential training tasks $\{\mathcal{D}_1, \dots, \mathcal{D}_T\}$, where $\mathcal{D}_\tau = \{(\mathbf{X}_\tau, \mathbf{Y}_\tau)\}$ denotes the training dataset of task τ . For each task $\tau \in \{1, \dots, T\}$, we denote the network parameters by $\{\mathbf{W}_{\tau,l}\}_{l=1}^L$, where $\mathbf{W}_{\tau,l}$ represents the weight matrix of the l -th layer. For notational simplicity, we omit the layer index when it is clear from the context and write $\mathbf{W}_\tau$ to denote the network weights of task τ .

Singular value decomposition. Let $\mathbf{W} \in \mathbb{R}^{n \times d}$ be a

matrix of rank $r \leq \min(n, d)$. Its SVD can be written as:

$$\mathbf{W} = \mathbf{U}\Sigma\mathbf{V}^\top = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top, \quad (1)$$

where $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_r] \in \mathbb{R}^{n \times r}$ and $\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_r] \in \mathbb{R}^{d \times r}$ have orthonormal columns, and $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{r \times r}$ contains the nonzero singular values. Throughout this work, we assume the singular values are ordered as $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$. We denote by $\sigma_i(\mathbf{W})$ the i -th singular value of $\mathbf{W}$.

SVD on convolutional filters. While SVD is naturally defined for 2D matrices, a convolutional filter is typically represented as a 4D tensor, which makes direct decomposition inapplicable. To address this, we reshape the tensor into a matrix before applying SVD. Specifically, for a convolutional layer with C_i input channels, C_o output channels, and a $K \times K$ kernel, the filter tensor $\mathcal{W} \in \mathbb{R}^{C_o \times C_i \times K \times K}$ is reshaped into a matrix $\mathbf{W} \in \mathbb{R}^{(C_i K^2) \times C_o}$.

3. Revisiting Gradients in Continual Learning

In this section, we revisit the singular value spectrum of gradients in three canonical classes of CL methods: ER, MAS, and GPM.

3.1. Singular Value Spectrum of Gradients

Examining gradients provides direct insight into optimization dynamics. Recently, [Lewandowski et al. \(2025\)](#) link rank reduction of the gradient matrix to a loss of plasticity, showing that reduced rank limits the diversity of gradient directions available for parameter updates. However, [Lewandowski et al. \(2025\)](#) focus primarily on methods that enhance plasticity and did not analyze the gradient properties induced by methods that mitigate forgetting. In this paper, we bridge that gap by directly examining the singular value spectrum of gradients, which offers a geometrically interpretable and numerically stable characterization of gradient structure. Below, we focus on three canonical classes of CL methods, ER, MAS and GPM, to investigate the limitations that hinder further progress.

We conduct experiments with ER, MAS and GPM on the CIFAR-100 benchmark using an AlexNet backbone to revisit the singular value spectrum of the gradient across tasks. In Figure 1, the spectra for ER and MAS exhibit a decay: a small set of dominant singular values is followed by a long tail of near-zero singular values. We observe a special case in GPM: the spectrum displays a pronounced ‘‘cliff’’ that separates a few relatively large singular values from a bulk of near-zero values, and this cliff appears at smaller singular value indices as the number of tasks increases. We refer to these behaviors as *singular value collapse*: only a

small subset of singular values remains substantial, while the remainder are comparatively small and often close to zero. The singular value spectrum of GPM differs from the trend in ER and MAS because GPM directly projects gradients to remove components overlapping with previous tasks, whereas ER and MAS implicitly shape the gradient’s spectral distribution. Additional plots are available in Appendix F.1. We will present a theoretical analysis in Section 3.2 that characterizes this phenomenon in GPM and generalizes it to orthogonal-projection-based methods.

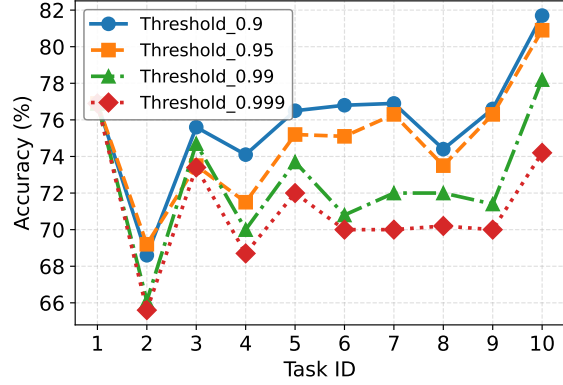


Figure 2. Accuracy curves under different GPM thresholds on the CIFAR-100 benchmark.

Next, we formally characterize singular value collapse. For any training step t on task τ under an arbitrary continual-learning method, denote the gradient by $\mathbf{G}_\tau^t$. Without loss of generality, let $\text{rank}(\mathbf{G}_\tau^t) = k$. Then, the singular value decomposition of $\mathbf{G}_\tau^t$ is

$$\mathbf{G}_\tau^t = \mathbf{U}_\tau^t \Sigma_\tau^t \mathbf{V}_\tau^{t\top} = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top. \quad (2)$$

Under singular value collapse, a small subset of singular values is substantially larger than the remainder. Without loss of generality, suppose Σ_τ^t contains r dominant singular values $\sigma_1, \dots, \sigma_r$ while the remaining $k - r$ singular values $\sigma_{r+1}, \dots, \sigma_k$ are comparatively small. From Equation (2) we therefore decompose the gradient $\mathbf{G}_\tau^t$ as

$$\mathbf{G}_\tau^t = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top + \sum_{j=r+1}^k \sigma_j \mathbf{u}_j \mathbf{v}_j^\top. \quad (3)$$

As observed in Figure 1, a long tail of near-zero singular values frequently appears. Although the associated singular vectors are present, their contributions are strongly suppressed by these near-zero singular values. As a result, the second term becomes negligible in practice, leaving the first term to dominate the gradient direction. Consequently, the parameter update components responsible for mitigating

forgetting depend primarily on the leading singular vectors $\mathbf{u}_1, \dots, \mathbf{u}_r$ and $\mathbf{v}_1, \dots, \mathbf{v}_r$. We refer to these as the *mitigating-forgetting directions*. In these methods, parameter updates are thus dominated by these directions, while the remaining directions contribute only marginally.

We have established that singular value collapse effectively suppresses a subset of parameter update directions. To connect this phenomenon with a loss of plasticity, we substantiate the impairment by presenting a concrete example. Because GPM operates directly on gradients, it can more readily control the degree of singular value collapse. In practice, GPM uses a projection threshold to determine the number of orthogonal basis vectors retained for gradient projection. Appendix F.2 provides the singular value plots for different threshold settings. We find that a larger threshold increases the number of retained basis vectors and thus intensifies singular value collapse. This intensification produces a greater number of near-zero singular values in the tail. As shown in Figure 2, the accuracy curve shifts downward as the threshold increases, which indicates that *more severe singular value collapse exacerbates the loss of plasticity*.

Discussions. Singular value analysis in CL has been studied in prior work, and here we clarify how our approach differs. Zhu et al. (2021) perform an eigen-decomposition of the old and new feature extractors and define the angle between the corresponding eigenvectors to capture representation shifts during incremental learning. Guo et al. (2022) compute the eigenvalues and eigenvectors of the learned representations to compare the overall representation quality of two different representation learning methods.

3.2. Singular Value Collapse Analysis

As discussed in Section 3.1, the singular value spectrum exhibits a cliff that separates a few relatively large singular values from a bulk of near-zero values, and this cliff shifts leftward as tasks accumulate. We generalize this phenomenon to orthogonal-projection-based methods and provide a theoretical analysis that explains its emergence.

Orthogonal-projection-based methods have achieved notable success in mitigating forgetting by updating the weight $\mathbf{W}_\tau$ for task τ along directions orthogonal to the subspace $\mathcal{X}_\tau$ spanned by the previous $\tau - 1$ tasks (Farajtabar et al., 2020; Saha et al., 2021; Lin et al., 2022; Yang et al., 2023a). Motivated by the theoretical analysis of Zheng et al. (2026), we present a unified formulation of orthogonal projection based methods.

For any $\tau \in \{1, \dots, T\}$, let $\mathcal{X}_\tau$ denote the subspace spanned by an orthonormal basis

$$\mathcal{B}_\tau = \{\mathbf{b}_{1,1}, \dots, \mathbf{b}_{1,n_1}, \dots, \mathbf{b}_{\tau-1,1}, \dots, \mathbf{b}_{\tau-1,n_{\tau-1}}\}.$$

The subspace $\mathcal{X}_{\tau+1}$ is obtained by augmenting $\mathcal{X}_\tau$ with a

set of new orthonormal vectors $\{\mathbf{b}_{\tau,1}, \dots, \mathbf{b}_{\tau,n_\tau}\}$, so that

$$\mathcal{X}_{\tau+1} = \text{span}(\mathcal{B}_\tau \cup \{\mathbf{b}_{\tau,1}, \dots, \mathbf{b}_{\tau,n_\tau}\}).$$

The procedure used to select the new orthonormal vectors depends on the specific orthogonal-projection method. Saha et al. (2021) obtain these vectors as the leading left singular vectors of the per-layer representation matrix computed after each task. In contrast, Farajtabar et al. (2020) store a set of model gradient vectors and constructs an orthonormal basis via the Gram–Schmidt procedure.

Let $P_{\mathcal{X}_\tau}$ denote the orthogonal projector onto the subspace $\mathcal{X}_\tau$ and let $P_{\mathcal{X}_\tau}^\perp$ denote the projector onto its orthogonal complement. The gradient $\mathbf{G}_\tau^t$ at training step t for task τ is then replaced by its projection onto the orthogonal complement,

$$\tilde{\mathbf{G}}_\tau^t = P_{\mathcal{X}_\tau}^\perp(\mathbf{G}_\tau^t),$$

and $\tilde{\mathbf{G}}_\tau^t$ is used for the subsequent parameter update.

Building on the unified formulation for orthogonal-projection-based methods established above, we provide a theoretical analysis of the singular value cliff phenomenon described in Section 3.1. The proof of Theorem 3.1 is deferred to Appendix C.

Theorem 3.1. *Let $\mathbf{G} \in \mathbb{R}^{n \times d}$ be a matrix, let $\mathcal{X}$ be an m -dimensional subspace of $\mathbb{R}^n$ spanned by the orthogonal basis $\{\mathbf{b}_1, \dots, \mathbf{b}_m\}$, and let $P_{\mathcal{X}}^\perp$ be the orthogonal projection operator onto the orthogonal complement of $\mathcal{X}$. Then, the inequality $\text{rank}(P_{\mathcal{X}}^\perp(\mathbf{G})) \leq \text{rank}(\mathbf{G})$ holds. Moreover, if the intersection of the column space of $\mathbf{G}$ and $\mathcal{X}$ is non-trivial, then the rank strictly decreases, i.e., $\text{rank}(P_{\mathcal{X}}^\perp(\mathbf{G})) < \text{rank}(\mathbf{G})$.*

Remark 3.2. Theorem 3.1 shows that projecting a gradient matrix onto the orthogonal complement of a subspace yields a matrix whose rank is non-increasing. Moreover, if the column space of the gradient has a nontrivial intersection with the projected subspace, the rank decreases strictly. In orthogonal-gradient-based methods, the constraint subspace typically expands as training progresses. This expansion increases the probability of such intersections and hence produces progressive rank reduction. In particular, when rank reduction occurs, a tail of singular values becomes exactly zero, and the number of zero singular values tends to grow as tasks accumulate. This mechanism explains why the spectral cliff begins to appear at task 2 and why the cliff shifts leftward as the number of tasks increases in GPM (see Figure 1). In the extreme case where the constraint subspace spans the entire parameter space, the orthogonal projector $P_{\mathcal{X}_n}^\perp$ annihilates every matrix, so $\text{rank}(P_{\mathcal{X}_n}^\perp(\mathbf{G})) = 0$. Consequently, the gradient vanishes and the model loses plasticity entirely.

4. Methodology

In this section, we introduce our method PAPO, which mitigates the negative impact of singular value collapse as discussed in Section 4.1. We also present efficient computational strategies for its components in Sections 4.2 and 4.3.

4.1. How can stability and plasticity be improved in CL?

As discussed in Section 3, most CL methods that mitigate forgetting suffer from singular value collapse, whereby a large fraction of singular directions contribute little or nothing to parameter updates. This phenomenon impairs model plasticity. Our objective is therefore to activate the remaining singular directions during optimization while preserving the dominant influence of the mitigating-forgetting directions. Accordingly, we pose the following question:

Q: How can the mitigating-forgetting directions retain their dominant role in updates while the remaining singular directions are activated?

To address **Q**, we adopt two design desiderata that constrain how the gradient may be modified:

1. The singular vectors $\{\mathbf{u}_i\}_{i=1}^k$ and $\{\mathbf{v}_i\}_{i=1}^k$ remain unchanged after the modification.
2. The singular values associated with the mitigating-forgetting directions should remain larger than those of the remaining directions after the modification.

Remark 4.1. Condition 1 is a conservative design choice intended to preserve the original update *directions*. The singular vectors $\{\mathbf{u}_i\}$ and $\{\mathbf{v}_i\}$ define the principal left/right directions along which parameter updates act. Altering these vectors effectively changes the update basis and can therefore modify which components of previous tasks are reinforced or suppressed. Because our goal is to *activate* previously suppressed directions while retaining the mechanisms that mitigate forgetting, preserving the singular vectors prevents unintended perturbations of prior information.

Remark 4.2. Condition 2 guarantees that the modification does not demote the mitigating-forgetting directions. Concretely, if the post-modification singular values corresponding to the mitigating directions remain larger than those of the previously suppressed directions, the original dominant directions will continue to carry the larger update coefficients. This preserves stability while increasing plasticity by lifting near-zero singular values of the residual directions.

Motivated by Remark 4.1 and Remark 4.2, we introduce the polar operator as follows.

Definition 4.3 (Polar operator). For any matrix $\mathbf{M}$ with the singular value decomposition $\mathbf{M} = \mathbf{U}\Sigma\mathbf{V}^\top$, the polar

operator of $\mathbf{M}$ is defined as

$$\text{polar}(\mathbf{M}) := \mathbf{U}\mathbf{V}^\top.$$

In fact, $\text{polar}(\mathbf{M})$ denotes the closest semi-orthogonal matrix to $\mathbf{M}$ in the Frobenius norm sense (Higham, 2008). In particular, if $\mathbf{M}$ is square, then $\text{polar}(\mathbf{M})$ is the orthogonal factor in the polar decomposition of $\mathbf{M}$.

Based on the polar operator, we can address **Q** as follows:

$$\mathbf{G}_\tau^t = \mathbf{G}_\tau^t + \lambda \cdot \text{polar}(\mathbf{G}_\tau^t) = \mathbf{U}_\tau^t (\Sigma_\tau^t + \lambda \mathbf{I}) \mathbf{V}_\tau^{t\top}, \quad (4)$$

where λ is a balancing coefficient. Under singular value collapse, we can rewrite Equation (4) as follows:

$$\mathbf{G}_\tau^t = \underbrace{\sum_{i=1}^r (\sigma_i + \lambda) u_i v_i^\top}_{\text{Mitigate Forgetting}} + \underbrace{\sum_{j=r+1}^k (\sigma_j + \lambda) u_j v_j^\top}_{\text{Improve Plasticity}} \quad (5)$$

As shown in Equation (5), applying the polar operator increases all singular values by a uniform magnitude λ . Consequently, the relative ordering of the singular values remains invariant compared to the original gradient. Crucially, this operation preserves the singular vectors, thereby maintaining the original update directions. Hence, **Q** is addressed by incorporating the polar operator: it activates previously neglected gradient directions without disrupting the primary directions responsible for mitigating forgetting.

However, computing $\text{polar}(\mathbf{G}_\tau^t)$ directly requires an SVD, which is computationally expensive if performed at every optimization step. In Section 4.2 we introduce an iterative algorithm that avoids SVD by approximating the polar factor using only matrix multiplications and additions. This is therefore well suited to efficient GPU implementation.

4.2. Computing the Polar Factor

Although $\text{polar}(\mathbf{G}_\tau^t)$ can be obtained directly from the SVD, standard SVD algorithms are prohibitively expensive for deep-learning applications and do not fully exploit the parallelism available on GPUs (Amsel et al., 2025). Numerous numerical schemes have therefore been developed to compute the polar factor, including Newton–Schulz iterations (Higham, 2008), Padé iterations (Higham, 1986), QWHD iterations (Nakatsukasa et al., 2010), and Zolotarev-based polar decompositions (Nakatsukasa & Freund, 2016). Many of these methods require explicit matrix inverses or QR decompositions, operations that limit efficient GPU acceleration. By contrast, the Newton–Schulz approach is based on a polynomial approximation of the matrix sign function and uses only matrix multiplications and additions. Therefore, it is particularly well suited to GPU implementation.

Newton–Schulz iterations. Set the initial iterate as $\mathbf{X}_0 = \mathbf{G}_\tau^t / \|\mathbf{G}_\tau^t\|_F$. For $k \geq 1$, the iterate $\mathbf{X}_k$ is updated as

$$\mathbf{X}_k = a\mathbf{X}_{k-1} + b(\mathbf{X}_{k-1}\mathbf{X}_{k-1}^\top)\mathbf{X}_{k-1} + c(\mathbf{X}_{k-1}\mathbf{X}_{k-1}^\top)^2\mathbf{X}_{k-1}, \quad (6)$$

where a , b and c are scalar coefficients. Convergence of the iteration in Equation (6) is promoted by choosing the coefficients so that the scalar polynomial

$$f(x) = ax + bx^3 + cx^5$$

has a fixed point at $x = 1$ with favorable local contraction properties.

Jordan method. Jordan et al. (2024) use a heuristic numerical search to propose a fixed-point iteration with coefficients $a = 3.4445$, $b = -4.7750$, and $c = 2.0315$. However, this iteration does not fully converge to $\text{polar}(\mathbf{G}_\tau^t)$ and instead plateaus at an approximation error of approximately 0.3. Nevertheless, this method has been successfully employed in the MUON optimizer (Jordan et al., 2024).

Most methods based on the Newton–Schulz iteration employ fixed coefficients a , b , and c at each step. In practice, this choice often leads to slow convergence in the early iterations, particularly when the initial iterate $\mathbf{X}_0$ is far from $\text{polar}(\mathbf{G}_\tau^t)$. To address this issue, Polar Express (Amsel et al., 2025) has been proposed as an alternative iterative scheme. By selecting iteration-dependent coefficients rather than a single fixed set, Polar Express significantly accelerates convergence during the early stages of the iteration.

Polar Express. Polar Express adopts the same iterative form as in Equation (6), but the coefficients are updated at each iteration rather than kept fixed. In particular, Polar Express determines the coefficients a , b , and c by greedily solving a minimax optimization problem adapted to the evolving range of singular values at each step. The resulting iteration-dependent coefficients, which are pre-computed for efficiency, are summarized in Table 3 in Appendix D.

Why do we prefer Polar Express? Although the Jordan method has been successfully employed in the MUON optimizer, it may be less suitable as a general-purpose component for PAPO. Empirically, the Jordan iteration does not yield the exact polar factor $\mathbf{UV}^\top$, but instead produces an approximation of the form $\mathbf{USV}^\top$, where $\mathbf{S}$ is a diagonal matrix whose entries satisfy $\mathbf{S}_{ii} \sim \text{Uniform}(0.5, 1.5)$. However, in PAPO this nonuniform scaling may undesirably alter the relative magnitudes of singular directions, potentially weakening the dominance of the mitigating-forgetting directions after applying Equation (4). By contrast, Polar Express attains a closer approximation to $\text{polar}(\mathbf{G}_\tau^t)$ within a small number of iterations, producing a correction that is

closer to the identity on the dominant subspace. Because PAPO is intended as a broadly applicable plug-in for diverse continual learning methods, we adopt Polar Express as a safer and more stable choice in practice.

4.3. Plasticity Activation via Polar Operator

As discussed in Sections 4.1 and 4.2, we propose a method termed Plasticity Activation via Polar Operator (PAPO), which applies the polar operator to the gradient in order to enhance both stability and plasticity, as shown in Equation (4). To compute the polar factor in practice, we introduce the Polar Express approximation scheme, which replaces the use of SVD with a purely iterative procedure that relies only on matrix multiplications and additions.

To reduce computational overhead, we apply PAPO to the gradients periodically with an interval of s epochs. Specifically, the gradients are rectified using Equation (4) before the parameter update only when the current epoch index is a multiple of s ; otherwise, the original gradients are used without modification. To simplify the notation, $\text{polar}(\cdot)$ in our algorithm denotes the iterative approximation defined by Equation (6), with coefficients selected according to Table 3 in Appendix D. The complete training procedure is summarized in Algorithm 1.

Algorithm 1 Plasticity Activation via Polar Operator

Require: Task sequence $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$, balance coefficient $\lambda_1, \dots, \lambda_N$, polar interval s , training epochs E .

- 1: **Initialize:** Model parameters $\theta \leftarrow \theta_0$
- 2: **for** Task ID $\tau = 1$ **to** N **do**
- 3: **for** epoch $t = 1$ **to** E **do**
- 4: $\mathbf{G}_\tau^t \leftarrow \nabla_{\theta} \mathcal{L}(\theta_\tau^{t-1})$
- 5: **if** $t \bmod s = 0$ **then**
- 6: $\mathbf{G}_\tau^t \leftarrow \mathbf{G}_\tau^t + \lambda_\tau \text{polar}(\mathbf{G}_\tau^t)$
- 7: **end if**
- 8: $\theta_\tau^t \leftarrow \theta_\tau^{t-1} - \eta \mathbf{G}_\tau^t$
- 9: **end for**
- 10: **end for**
- 11: **Return** θ

Next, we summarize the main advantages of our algorithm:

- PAPO is a plug-in module that can be seamlessly integrated into existing CL algorithms without modifying their loss functions or model architectures.
- PAPO relies exclusively on matrix multiplications and additions, making it well suited for efficient implementation on modern GPU hardware.

5. Experiments

In this section, we conduct extensive experiments to demonstrate the effectiveness of PAPO. The implementation details and additional experiments are provided Appendix E.

Table 1. ACC and BWT over all tasks on different datasets. Higher ACC and BWT indicate better generalization and less forgetting. All results are reproduced by us and averaged over 5 runs.

Method	P-MNIST (10 tasks)		CIFAR-100 (10 tasks)		5-Datasets (5 tasks)		MiniImageNet (20 tasks)	
	ACC (%) $\uparrow$	BWT $\uparrow$	ACC (%) $\uparrow$	BWT $\uparrow$	ACC (%) $\uparrow$	BWT $\uparrow$	ACC (%) $\uparrow$	BWT $\uparrow$
ER	87.24 $\pm$ 0.53	-0.11 $\pm$ 0.01	71.73 $\pm$ 0.63	-0.06 $\pm$ 0.01	88.31 $\pm$ 0.22	-0.04 $\pm$ 0.00	63.40 $\pm$ 2.94	-0.06 $\pm$ 0.01
+PAPO	88.18 $\pm$ 0.42	-0.10 $\pm$ 0.00	74.07 $\pm$ 0.48	-0.05 $\pm$ 0.01	90.30 $\pm$ 0.33	-0.04 $\pm$ 0.00	72.41 $\pm$ 1.17	-0.03 $\pm$ 0.01
MAS	86.98 $\pm$ 0.84	-0.04 $\pm$ 0.01	67.96 $\pm$ 0.44	-0.05 $\pm$ 0.00	88.73 $\pm$ 0.79	-0.04 $\pm$ 0.01	57.31 $\pm$ 2.64	-0.06 $\pm$ 0.02
+PAPO	88.15 $\pm$ 0.26	-0.03 $\pm$ 0.01	72.55 $\pm$ 0.47	-0.05 $\pm$ 0.00	90.72 $\pm$ 0.18	-0.00 $\pm$ 0.00	62.07 $\pm$ 1.71	-0.02 $\pm$ 0.01
GPM	93.18 $\pm$ 0.14	-0.03 $\pm$ 0.00	72.04 $\pm$ 0.18	-0.00 $\pm$ 0.00	91.12 $\pm$ 0.20	-0.01 $\pm$ 0.00	60.20 $\pm$ 1.98	-0.04 $\pm$ 0.02
+PAPO	94.00 $\pm$ 0.27	-0.03 $\pm$ 0.00	72.62 $\pm$ 0.43	+0.00 $\pm$ 0.00	92.00 $\pm$ 0.07	-0.01 $\pm$ 0.00	69.10 $\pm$ 1.48	+0.00 $\pm$ 0.00
TRGP	96.34 $\pm$ 0.11	-0.01 $\pm$ 0.00	74.46 $\pm$ 0.32	-0.01 $\pm$ 0.00	93.56 $\pm$ 0.10	-0.00 $\pm$ 0.00	61.78 $\pm$ 0.60	-0.01 $\pm$ 0.01
+PAPO	96.73 $\pm$ 0.10	-0.01 $\pm$ 0.00	75.65 $\pm$ 0.17	+0.00 $\pm$ 0.00	94.04 $\pm$ 0.13	-0.00 $\pm$ 0.00	70.97 $\pm$ 0.82	-0.00 $\pm$ 0.00

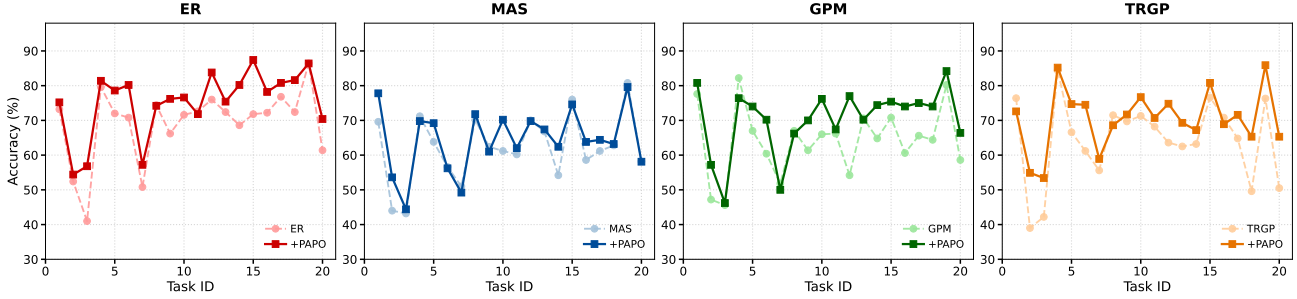


Figure 3. Performance comparison on the MiniImageNet dataset.

5.1. Experimental Setup

Datasets and network architectures. For *vision* tasks we follow the dataset and architecture configurations of Saha et al. (2021). We evaluate on four standard CL benchmarks: Permuted MNIST (PMNIST) (Lecun et al., 2002), 10-split CIFAR-100 (Krizhevsky, 2009), 20-split MiniImageNet (Vinyals et al., 2016), and the 5-Datasets sequence (Ebrahimi et al., 2020). For PMNIST we use a fully connected network with two hidden layers of 100 units each. For Split CIFAR-100 we adopt a 5-layer AlexNet. For Split MiniImageNet and the 5-Datasets sequence we use a reduced ResNet-18. For *language* tasks, we follow the setup of Liang et al. (2026) and evaluate on the SuperNI benchmark (Wang et al., 2022b), which comprises 15 tasks and two task orders (Order 1 and Order 2). For SuperNI benchmark we use Llama-2-7B (Touvron et al., 2023) as the backbone model. Further dataset and implementation details are provided in Appendix E.1.

Baselines. For vision tasks, we evaluate PAPO when applied to representative methods from three categories of CL: a replay-based method (ER with reservoir sampling (Chaudhry et al., 2019b)), a regularization-based method (MAS (Aljundi et al., 2018)), and orthogonal-projection-based methods (GPM (Saha et al., 2021) and TRGP (Lin et al., 2022)). For language experiments, we apply PAPO to InfLoRA (Liang & Li, 2024) and GainLoRA (Liang et al., 2026). Detailed hyperparameter settings are reported in

Appendix E.2.

Evaluation metrics. For a fair comparison, we adopt the same evaluation metrics as in Saha et al. (2021) and Liang et al. (2026). We report three standard metrics: average accuracy (ACC), backward transfer (BWT) and forgetting (FT). For vision tasks we report ACC and BWT; for language tasks we report ACC and FT. Let $A_{t,i}$ denote the test accuracy on task i after training up to task t , and let T be the total number of tasks. The metrics are defined as: $ACC = \frac{1}{T} \sum_{i=1}^T A_{T,i}$, $BWT = \frac{1}{T-1} \sum_{i=1}^{T-1} (A_{T,i} - A_{i,i})$, $FT = \frac{1}{T-1} \sum_{i=1}^{T-1} (\max_{t \in \{1, \dots, T-1\}} A_{t,i} - A_{T,i})$.

Other Experiments. We also conduct experiments in the non-CL setting in Appendix F.5, in the online-CL setting in Appendix F.6, and include comparisons with additional methods in Appendix F.7.

5.2. Performance Comparison

5.2.1. RESULTS ON VISION TASKS

ACC and BWT. We first investigate how incorporating PAPO into CL methods affects ACC and BWT. As shown in Table 1, PAPO yields consistent improvements across all baselines. In particular, averaged over the four benchmarks, incorporating PAPO into ER, MAS, GPM, and TRGP results in accuracy gains of 3.52%, 3.18%, 2.80%, and 2.91%, with corresponding BWT increases of 0.01, 0.02, 0.01, and 0.01, respectively. The effectiveness of PAPO is espe-

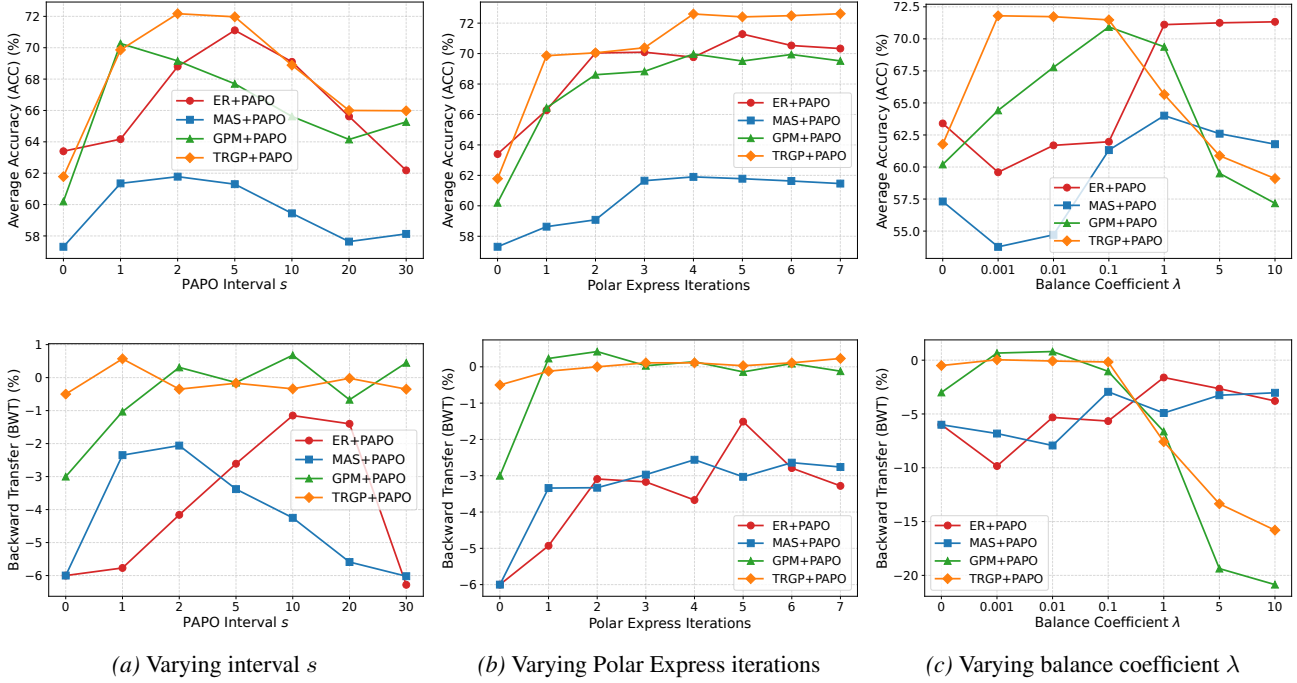


Figure 4. Ablation study of PAPO hyperparameters on the MiniImageNet dataset. The top row reports the ACC, while the bottom row reports the BWT. The results illustrate the effect of (a) the PAPO application interval s , (b) the number of Polar Express iterations, and (c) the balance coefficient λ on model performance.

cially pronounced on the MiniImageNet benchmark, where it improves the accuracy of ER, MAS, GPM, and TRGP by 9.01%, 4.76%, 8.90%, and 9.19%, respectively, while simultaneously alleviating forgetting with BWT improvements of 0.03, 0.04, 0.04, and 0.01. We note that these datasets are widely studied benchmarks in CL. Therefore, achieving consistent improvements on them highlights the practical significance of PAPO.

Plasticity. To assess the plasticity of CL methods, we examine the test accuracy on each task after training on successive tasks on the MiniImageNet benchmark. As shown in Figure 3, across all four subfigures the curves with darker colors and square markers generally lie above the corresponding curves with lighter colors and circular markers. This observation suggests that incorporating PAPO into ER, MAS, GPM, and TRGP tends to improve test accuracy on individual tasks, thereby enhancing model plasticity.

We also compare the gradient singular value spectra produced by ER, MAS and GPM with and without PAPO in Appendix F.3, highlighting that incorporating PAPO alleviates singular value collapse.

5.2.2. RESULTS ON LANGUAGE TASKS

As shown in Table 2, averaged over Order 1 and Order 2, incorporating PAPO into InfLoRA and GainLoRA yields accuracy gains of 3.37% and 1.24%, respectively, with cor-

Table 2. ACC and FT on the SuperNI benchmark (Order 1 and Order 2) using the Llama-2-7B model. Higher ACC and lower FT indicate better generalization and less forgetting.

Methods	Order 1		Order 2	
	ACC $\uparrow$	FT $\downarrow$	ACC $\uparrow$	FT $\downarrow$
InfLoRA	42.93	11.23	39.94	15.00
+PAPO	48.08	6.29	41.53	13.51
GainLoRA	51.27	2.84	50.17	4.71
+PAPO	52.47	2.09	51.45	2.69

responding reductions in forgetting of 3.22% and 1.39%. We further analyze the computational cost introduced by PAPO in Appendix F.4, highlighting that PAPO is computationally efficient.

5.3. Ablation Study

Interval s . Figure 4a illustrates the effect of periodically applying PAPO with interval s (in epochs) on ACC and BWT. Applying PAPO at every epoch, that is $s = 1$, increases computational cost and does not consistently yield the best performance. In contrast, a large interval s fails to activate the previously suppressed directions sufficiently often; in this regime, PAPO behaves more like a sporadic perturbation that abruptly alters the update direction. As a result, performance degrades noticeably when $s \geq 10$. In particular, when $s = 30$, applying PAPO to ER even

leads to worse performance than the baseline without PAPO. Empirically on miniImageNet, an intermediate interval s in the range of 1–5 provides a favorable trade-off between computational overhead and improvements in both ACC and BWT across the evaluated methods.

Polar Express iterations. Figure 4b illustrates the effect of the number of Polar Express iterations used to approximate $\text{polar}(\mathbf{G}_T^t)$ on ACC and BWT. Both metrics improve rapidly in the first few iterations and then plateau. Notably, even a single iteration yields a clear performance gain. In practice, 2–4 iterations are sufficient to capture most of the benefits while maintaining computational efficiency. This trend aligns with the expectation that a small number of iteration-dependent updates can substantially enhance performance compared with a zero-iteration baseline.

Balance coefficient λ . The balance coefficient λ directly affects the singular values of the gradient after applying PAPO, as each singular value σ_i is shifted to $\sigma_i + \lambda$. Consequently, λ controls the relative strength of the dominant mitigating-forgetting directions in the updated gradient. If λ is excessively large, it can dominate the original spectrum and thereby disturb these dominant directions. This effect is more pronounced when the original singular values are small, as is often the case for orthogonal-projection-based methods, since in this regime $\sigma_i + \lambda \approx \lambda$ for all i . As shown in Figure 4c, when $\lambda < 1$, both ACC and BWT for TRGP and GPM improve; however, when $\lambda \geq 1$, their performance degrades, and at $\lambda = 10$ these methods exhibit severe forgetting. This observation corroborates Remark 4.2. If the dominant role of the mitigating-forgetting directions is disrupted, the model’s stability is compromised and forgetting may increase.

6. Conclusion

In this work, we first revisit the singular value spectra of gradients in three representative CL algorithms that are designed to mitigate forgetting, and we show that these methods consistently exhibit singular value collapse. We then propose Plasticity Activation via the Polar Operator (PAPO), a mechanism that preserves the forgetting-mitigation properties of the base methods while improves plasticity. Finally, we evaluate PAPO on both vision and language benchmarks and demonstrate that incorporating PAPO into existing CL methods yields higher accuracy and reduced forgetting.

Acknowledgment

This work is supported by National Key R&D Projects (NO. 2024YFC3307100), NSFC Grant (No. 62576364), Guangdong Basic and Applied Basic Research Foundation (2026B1515020071), Shenzhen Basic Research Project (Natural Science Foundation) Basic Re-

search Key Project (NO. JCYJ20241202124430041), and Shenzhen Science and Technology Program (NO.SYSRD20250529113401002). The computation is completed in the HPC Platform of Huazhong University of Science and Technology.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Aljundi, R., Babiloni, F., Elhoseiny, M., Rohrbach, M., and Tuytelaars, T. Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European conference on computer vision (ECCV)*, pp. 139–154, 2018.
- Amsel, N., Persson, D., Musco, C., and Gower, R. M. The polar express: Optimal matrix sign methods and their application to the muon algorithm. *arXiv preprint arXiv:2505.16932*, 2025.
- Bian, A., Li, W., Yuan, H., Yu, C., Wang, M., Zhao, Z., Lu, A., Ji, P., and Feng, T. Make continual learning stronger via c-flat. *Advances in Neural Information Processing Systems*, 37:7608–7630, 2024.
- Buzzega, P., Boschini, M., Porrello, A., Abati, D., and Calderara, S. Dark experience for general continual learning: a strong, simple baseline. *Advances in neural information processing systems*, 33:15920–15930, 2020.
- Carpenter, G. A. and Grossberg, S. Art 2: Self-organization of stable category recognition codes for analog input patterns. *Applied optics*, 26(23):4919–4930, 1987.
- Chaudhry, A., Ranzato, M., Rohrbach, M., and Elhoseiny, M. Efficient lifelong learning with a-GEM. In *International Conference on Learning Representations*, 2019a. URL https://openreview.net/forum?id=Hkf2_sC5FX.
- Chaudhry, A., Rohrbach, M., Elhoseiny, M., Ajanthan, T., Dokania, P., Torr, P., and Ranzato, M. Continual learning with tiny episodic memories. In *Workshop on Multi-Task and Lifelong Reinforcement Learning*, 2019b.
- Chen, Q., Shui, C., Han, L., and Marchand, M. On the stability-plasticity dilemma in continual meta-learning: Theory and algorithm. *Advances in Neural Information Processing Systems*, 36:27414–27468, 2023.
- Dhar, P., Singh, R. V., Peng, K.-C., Wu, Z., and Chellappa, R. Learning without memorizing. In *Proceedings of the*

- IEEE/CVF conference on computer vision and pattern recognition*, pp. 5138–5146, 2019.
- Ebrahimi, S., Meier, F., Calandra, R., Darrell, T., and Rohrbach, M. Adversarial continual learning. In *European conference on computer vision*, pp. 386–402. Springer, 2020.
- Elsayed, M. and Mahmood, A. R. Addressing loss of plasticity and catastrophic forgetting in continual learning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=sKPzAXoylB>.
- Farajtabar, M., Azizan, N., Mott, A., and Li, A. Orthogonal gradient descent for continual learning. In *International conference on artificial intelligence and statistics*, pp. 3762–3773. PMLR, 2020.
- Feng, T., Li, W., Zhu, D., Yuan, H., Zheng, W., Zhang, D., and Tang, J. Zeroflow: Overcoming catastrophic forgetting is easier than you think. In *International Conference on Machine Learning*, pp. 16808–16824. PMLR, 2025.
- Foret, P., Kleiner, A., Mobahi, H., and Neyshabur, B. Sharpness-aware minimization for efficiently improving generalization. *arXiv preprint arXiv:2010.01412*, 2020.
- Fostiropoulos, I., Zhu, J., and Itti, L. Batch model consolidation: A multi-task model consolidation framework. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 3664–3676, 2023.
- Gao, R. and Liu, W. Ddgr: Continual learning with deep diffusion-based generative replay. In *International Conference on Machine Learning*, pp. 10744–10763. PMLR, 2023.
- Grossberg, S. Adaptive resonance theory: How a brain learns to consciously attend, learn, and recognize a changing world. *Neural Networks*, 37:1–47, 2013.
- Guo, Y., Liu, B., and Zhao, D. Online continual learning through mutual information maximization. In *International conference on machine learning*, pp. 8109–8126. PMLR, 2022.
- Guo, Y., Liu, B., and Zhao, D. Dealing with cross-task class discrimination in online continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 11878–11887, 2023.
- Higham, N. J. Computing the polar decomposition—with applications. *SIAM Journal on Scientific and Statistical Computing*, 7(4):1160–1174, 1986.
- Higham, N. J. *Functions of matrices: theory and computation*. SIAM, 2008.
- Izmailov, P., Podoprikin, D., Garipov, T., Vetrov, D., and Wilson, A. G. Averaging weights leads to wider optima and better generalization. *arXiv preprint arXiv:1803.05407*, 2018.
- Jordan, K., Jin, Y., Boza, V., Jiacheng, Y., Cecista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks. URL <https://kellerjordan.github.io/posts/muon>, 6, 2024.
- Kim, J., Cho, H., Kim, J., Tiruneh, Y. Y., and Baek, S. Sddgr: Stable diffusion-based deep generative replay for class incremental object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 28772–28781, 2024.
- Kim, S., Noci, L., Orvieto, A., and Hofmann, T. Achieving a better stability-plasticity trade-off via auxiliary networks in continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 11930–11939, 2023.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.
- Kong, Y., Liu, L., Wang, Z., and Tao, D. Balancing stability and plasticity through advanced null space in continual learning. In *European Conference on Computer Vision*, pp. 219–236. Springer, 2022.
- Krizhevsky, A. Learning multiple layers of features from tiny images. *Technical report, University of Tront*, 2009.
- Lecun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 2002.
- Lewandowski, A., Bortkiewicz, M., Kumar, S., Gyorgy, A., Schuurmans, D., Ostaszewski, M., and C Machado, M. Learning continually by spectral regularization. In *International Conference on Learning Representations*, volume 2025, pp. 34504–34530, 2025.
- Li, W., Yuan, H., Zhao, Z., Kang, B., Liu, Z., and Feng, T. A faster path to continual learning. *arXiv preprint arXiv:2604.11064*, 2026.
- Li, Z. and Hoiem, D. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017.
- Liang, Y.-S. and Li, W.-J. Inflora: Interference-free low-rank adaptation for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 23638–23647, 2024.

- Liang, Y.-S., Chen, J.-R., and Li, W.-J. Gated integration of low-rank adaptation for continual learning of large language models. *Advances in Neural Information Processing Systems*, 38:68659–68689, 2026.
- Lin, S., Yang, L., Fan, D., and Zhang, J. Trgp: Trust region gradient projection for continual learning. In *The Tenth International Conference on Learning Representations*, 2022.
- Lopez-Paz, D. and Ranzato, M. Gradient episodic memory for continual learning. *Advances in neural information processing systems*, 30, 2017.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Lu, A., Yuan, H., Feng, T., and Sun, Y. Rethinking the stability-plasticity trade-off in continual learning from an architectural perspective. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J. (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 40888–40902. PMLR, 13–19 Jul 2025a. URL <https://proceedings.mlr.press/v267/lu25t.html>.
- Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., and Zhu, J. Dpm-solver++: Fast solver for guided sampling of diffusion probabilistic models. *Machine Intelligence Research*, 22 (4):730–751, 2025b.
- McCloskey, M. and Cohen, N. J. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*, volume 24, pp. 109–165. Elsevier, 1989.
- Nakatsukasa, Y. and Freund, R. W. Computing fundamental matrix decompositions accurately via the matrix sign function in two iterations: The power of zolotarev’s functions. *siam REVIEW*, 58(3):461–493, 2016.
- Nakatsukasa, Y., Bai, Z., and Gygi, F. Optimizing halley’s iteration for computing the matrix polar decomposition. *SIAM Journal on Matrix Analysis and Applications*, 31 (5):2700–2720, 2010.
- Riemer, M., Cases, I., Ajemian, R., Liu, M., Rish, I., Tu, Y., and Tesauro, G. Learning to learn without forgetting by maximizing transfer and minimizing interference. *arXiv preprint arXiv:1810.11910*, 2018.
- Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., and Hadsell, R. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.
- Saha, G., Garg, I., and Roy, K. Gradient projection memory for continual learning. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=3A0j0RCNC2>.
- Serra, J., Suris, D., Miron, M., and Karatzoglou, A. Overcoming catastrophic forgetting with hard attention to the task. In *International conference on machine learning*, pp. 4548–4557. PMLR, 2018.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Vinyals, O., Blundell, C., Lillicrap, T., Kavukcuoglu, K., and Wierstra, D. Matching networks for one shot learning. In *Proceedings of the 30th International Conference on Neural Information Processing Systems*, pp. 3637–3645, 2016.
- Wang, F.-Y., Zhou, D.-W., Liu, L., Ye, H.-J., Bian, Y., Zhan, D.-C., and Zhao, P. Beef: Bi-compatible class-incremental learning via energy-based expansion and fusion. In *The eleventh international conference on learning representations*, 2022a.
- Wang, M., Michel, N., Xiao, L., and Yamasaki, T. Improving plasticity in online continual learning via collaborative learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 23460–23469, 2024a.
- Wang, Y., Mishra, S., Alipoormolabashi, P., Kordi, Y., Mirzaei, A., Naik, A., Ashok, A., Dhanasekaran, A. S., Arunkumar, A., Stap, D., et al. Super-naturalinstructions: Generalization via declarative instructions on 1600+ nlp tasks. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 5085–5109, 2022b.
- Wang, Z., Zhan, Z., Gong, Y., Yuan, G., Niu, W., Jian, T., Ren, B., Ioannidis, S., Wang, Y., and Dy, J. Sparcl: Sparse continual learning on the edge. *Advances in Neural Information Processing Systems*, 35:20366–20380, 2022c.
- Wang, Z., Yang, E., Shen, L., and Huang, H. A comprehensive survey of forgetting in deep learning beyond continual learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024b.
- Wang, Z., Shen, L., Duan, T., Zhu, Y., Liu, T., Gao, M., and Tao, D. Release the potential of memory buffer in continual learning: A dynamic system perspective. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

- Yang, E., Shen, L., Wang, Z., Liu, S., Guo, G., and Wang, X. Data augmented flatness-aware gradient projection for continual learning. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 5630–5639, 2023a.
- Yang, E., Shen, L., Wang, Z., Liu, T., and Guo, G. An efficient dataset condensation plugin and its application to continual learning. *Advances in Neural Information Processing Systems*, 36:67625–67642, 2023b.
- Yang, E., Shen, L., Wang, Z., Liu, S., Guo, G., Wang, X., and Tao, D. Revisiting flatness-aware optimization in continual learning with orthogonal gradient projection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(5):3895–3907, 2025.
- Yang, E., Tang, A., Shen, L., Guo, G., Wang, X., Cao, X., and Zhang, J. Continual model merging without data: Dual projections for balancing stability and plasticity. *Advances in Neural Information Processing Systems*, 38: 39275–39305, 2026.
- Yoon, J., Yang, E., Lee, J., and Hwang, S. J. Lifelong learning with dynamically expandable networks. In *International Conference on Learning Representations*, 2018.
- Zenke, F., Poole, B., and Ganguli, S. Continual learning through synaptic intelligence. In *International conference on machine learning*, pp. 3987–3995. PMLR, 2017.
- Zhang, X., Wei, X., Hu, W., Wu, J., Wu, J., Zhang, W., Zhang, Z., Lei, Z., and Li, Q. A survey on personalized content synthesis with diffusion models. *Machine Intelligence Research*, 22(5):817–848, 2025.
- Zhao, W., Wang, S., Hu, Y., Zhao, Y., Qin, B., Zhang, X., Yang, Q., Xu, D., and Che, W. Sapt: A shared attention framework for parameter-efficient continual learning of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11641–11661, 2024.
- Zheng, G., Wang, P., Hu, S., Zheng, Q., and Shen, L. Understanding the dynamics of forgetting and generalization in continual learning via the neural tangent kernel. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=NE2yIxdolw>.
- Zhu, F., Cheng, Z., Zhang, X.-Y., and Liu, C.-I. Class-incremental learning via dual augmentation. *Advances in neural information processing systems*, 34:14306–14318, 2021.

A. Related work

Mitigate forgetting. CL has achieved substantial empirical progress, with existing approaches broadly categorized into three families: (1) *Regularization-based methods*, which introduce explicit penalty terms to restrict updates on parameters important for previous tasks (Kirkpatrick et al., 2017; Zenke et al., 2017), or employ knowledge distillation by aligning the predictions of the current model (student) with those of the previous model (teacher) to mitigate forgetting (Li & Hoiem, 2017; Dhar et al., 2019; Fostiropoulos et al., 2023); (2) *Replay-based methods*, which either directly store data or use a diffusion model (Lu et al., 2025b; Zhang et al., 2025) to replay data from past tasks during training on new tasks (Chaudhry et al., 2019a; Riemer et al., 2018; Buzzega et al., 2020; Yang et al., 2023b; Gao & Liu, 2023; Kim et al., 2024; Wang et al., 2025), or retain gradient or feature information from previous tasks and enforce new updates to remain orthogonal to past gradients, thereby avoiding explicit data replay (Farajtabar et al., 2020; Saha et al., 2021; Lin et al., 2022; Yang et al., 2023a; 2025). (3) *Architecture-based methods* (Rusu et al., 2016; Yoon et al., 2018; Wang et al., 2022c), which dynamically expand or adapt the network architecture to preserve knowledge from earlier tasks. Recently, gradient modification-based methods (Bian et al., 2024; Li et al., 2026; Feng et al., 2025) have also played an important role in improving model performance in CL.

Stability–plasticity trade-off. The trade-off between stability and plasticity was famously framed by Carpenter & Grossberg (1987) as the *stability–plasticity dilemma*: a fundamental tension between retaining performance on past tasks and adapting to new ones. Recent work has proposed diverse strategies to address this dilemma in CL. We categorize these strategies into three classes: architecture-based methods, methods tailored to specific CL algorithms, and methods tailored to specific CL paradigms. (1) Architecture-based approaches decouple components or add auxiliary modules to reconcile stability and plasticity. For example, Wang et al. (2022a) propose an energy-based framework that mitigates conflicts in dynamic architectures by decoupling the training of independent modules while ensuring bidirectional compatibility. Kim et al. (2023) introduce Auxiliary Network Continual Learning (ANCL), which employs a dual-network design in which a primary network preserves stability and an auxiliary network promotes plasticity via a learnable regularizer. Lu et al. (2025a) show that depth tends to favor plasticity while width favors stability and they propose the DualArch framework to separate these objectives using two specialized lightweight networks. (2) Methods that target specific CL algorithms include Kong et al. (2022)’s Advanced Null Space (AdNS), which uses low-rank approximation to derive a null space for gradient projection, and Yang et al. (2026)’s data-free dual orthogonal projection scheme for continual model merging. (3) Approaches designed for particular CL paradigms include Chen et al. (2023), who develop an excess-risk bound together with a dynamic optimization algorithm for continual meta-learning, and Elsayed & Mahmood (2024) propose Utility-based Perturbed Gradient Descent (UPGD), a method for streaming CL that protects useful units with small updates while rejuvenating less useful ones via larger perturbations. These prior solutions often add architectural complexity or remain specialized to particular algorithms or scenarios. As a result they do not offer a broadly applicable remedy to the stability and plasticity dilemma. In contrast, PAPO is an architecture-free plug-in that can be incorporated seamlessly into a wide range of CL algorithms.

B. Limitations

In this work we propose PAPO, a general method to enhance plasticity in CL algorithms that mitigate forgetting. PAPO depends on a coefficient λ that must be set manually. Developing an adaptive scheme for λ would increase PAPO’s performance. In future work we plan to apply PAPO to multimodal large language models (MLLMs) and to integrate it with additional CL methods.

C. Proof of Theorem 3.1

Proof. First, we prove the non-strict inequality. Recall the rank inequality for matrix multiplication: for any matrices A and B , $\text{rank}(AB) \leq \min\{\text{rank}(A), \text{rank}(B)\}$. By setting $A = P_{\mathcal{X}_m}^\perp$ and $B = G$, we immediately have

$$\text{rank}(P_{\mathcal{X}_m}^\perp G) \leq \text{rank}(G). \quad (7)$$

Next, we prove the strict one. Assume the intersection of the column space of G (denoted as $\text{Col}(G)$) and $\mathcal{X}_m$ is non-trivial. This implies that there exists a non-zero vector $v \in \text{Col}(G) \cap \mathcal{X}_m$.

Since $v \in \text{Col}(G)$, there exists a coefficient vector $w \in \mathbb{R}^d$ such that $v = Gw$. Note that $w \notin \ker(G)$ because $Gw = v \neq 0$.

Since $v \in \mathcal{X}_m$ and $P_{\mathcal{X}_m}^\perp$ projects onto the orthogonal complement of $\mathcal{X}_m$, we have $P_{\mathcal{X}_m}^\perp v = 0$.

Combining these, we have

$$(P_{\mathcal{X}_m}^\perp G)w = P_{\mathcal{X}_m}^\perp (Gw) = P_{\mathcal{X}_m}^\perp v = 0. \quad (8)$$

This shows that $w \in \ker(P_{\mathcal{X}_m}^\perp G)$. Consequently, the kernel of G is a proper subspace of the kernel of $P_{\mathcal{X}_m}^\perp G$ (i.e., $\ker(G) \subsetneq \ker(P_{\mathcal{X}_m}^\perp G)$).

By the Rank-Nullity Theorem, $\text{rank}(A) = d - \dim(\ker(A))$. Since $\dim(\ker(P_{\mathcal{X}_m}^\perp G)) > \dim(\ker(G))$, it follows that

$$\text{rank}(P_{\mathcal{X}_m}^\perp G) < \text{rank}(G). \quad (9)$$

The proof is complete. $\square$

D. Polar operator

Table 3. Coefficient settings for Polar Express. The reported values are taken from [Amsel et al. \(2025\)](#).

k	a	b	c
1	8.28721	-23.5959	17.3004
2	4.10706	-2.94785	0.544843
3	3.94869	-2.9089	0.551819
4	3.31842	-2.48849	0.510049
5	2.30065	-1.6689	0.418807
6	1.8913	-1.268	0.376804
7	1.875	-1.25	0.375
8	1.875	-1.25	0.375

E. Experimental Details and Additional Experiments

E.1. Dataset and Network Architecture

We summarize the datasets and network architectures used in our experiments. Details for vision tasks are provided in Section E.1.1, and details for language tasks are provided in Section E.1.2.

E.1.1. VISION TASK DATASETS AND NETWORK ARCHITECTURES

We follow the dataset and network architecture configurations of [Saha et al. \(2021\)](#).

Dataset. The PMNIST dataset consists of 10 sequential tasks generated by applying different random permutations to the pixels of the original MNIST images. The 10-Split CIFAR-100 dataset is constructed by partitioning the 100 classes of CIFAR-100 into 10 disjoint tasks with 10 classes per task. The 20-Split miniImageNet dataset is constructed by splitting the 100 classes of miniImageNet into 20 sequential tasks, each containing 5 classes. Finally, we evaluate on a sequence of five datasets, including CIFAR-10, MNIST, SVHN, notMNIST, and Fashion-MNIST, where classification on each dataset is treated as a separate task. No data augmentation is applied in any experiment.

Network Architecture. AlexNet-like architecture: This is the same architecture used by [\(Serra et al., 2018\)](#) with batch normalization added in each layer except the classifier layer. The network consists of 3 convolutional layers of 64, 128, and 256 filters with 4×4 , 3×3 , and 2×2 kernel sizes, respectively, plus two fully connected layers of 2048 units each. Rectified linear units is used as activations, and 2×2 max-pooling after the convolutional layers. Dropout of 0.2 is used for the first two layers and 0.5 for the rest. Reduced ResNet18 architecture: This is the similar architecture used by [\(Lopez-Paz & Ranzato, 2017\)](#). For miniImageNet experiment, we use convolution with stride 2 in the first layer. For both miniImageNet and 5-Datasets experiments we replace the 4×4 average-pooling before classifier layer with 2×2 average-pooling. All the networks use ReLU in the hidden units and softmax with cross entropy loss in the final layer.

E.1.2. LANGUAGE TASK DATASETS

We follow the dataset and network architecture configurations of [Liang et al. \(2026\)](#).

Dataset. We use the SuperNI benchmark as our language CL benchmark. SuperNI comprises a diverse set of language tasks, including dialogue generation, information extraction, question answering, summarization, and sentiment analysis. Following the protocol of Zhao et al. (2024), we select three tasks from each category, resulting in a total of 15 tasks. These tasks are organized into two different task sequences with distinct orders, referred to as Order 1 and Order 2.

E.2. Hyperparameter settings

In this section we list the hyperparameter settings for vision (Table 4) and language (Table 5) experiments. For a fair comparison, we adopt the original hyperparameter configurations used by Saha et al. (2021) and Lin et al. (2022) for the vision benchmarks and by Liang et al. (2026) for the language benchmarks.

Following Liang et al. (2026), language tasks are implemented with instruction tuning and optimized using AdamW (Loshchilov & Hutter, 2019). We incorporate LoRA adapters into the query and value projections of the multi-head attention in each Transformer block. The global batch size is set to 32 and each task is trained for 50 epochs. For GainLoRA, we follow Liang et al. (2026) and use InfLoRA to update the newly introduced LoRA branch.

Table 4. Hyperparameter settings for vision tasks. λ_0 denotes the PAPO coefficient for the first task, λ denotes the PAPO coefficient for tasks starting from the second one. The scale factor γ controls a geometric decay of the PAPO coefficient: for task $t \geq 2$ the coefficient is λ/γ^{t-2} . Experiments were run with seeds 1–5 and the reported results are averaged.

Setting	Hyperparameter	P-MNIST	CIFAR-100	5-Datasets	MiniImageNet
Shared	Task nums	10	10	5	20
	Network	MLP	AlexNet	ResNet-18	ResNet-18
	Epochs	5	200	100	100
	Learning rate	0.01	0.05	0.1	0.1
	Batch size	10	64	64	64
ER	Buffer size	1000	2000	3000	500
	Interval	2	5	5	5
	λ_0	0.01	0.5	1	5
	λ	0.01	0.5	1	5
	Scale factor γ	1	1	5	1
MAS	MAS coefficient	1	1	1	1
	Interval	5	5	2	2
	λ_0	10	5	5	10
	λ	0.001	5	5	10
	Scale factor γ	1	1	1	1
GPM	Interval	2	2	2	2
	λ_0	0.1	5	1	5
	λ	0.0001	0.005	0.05	0.05
	Scale factor γ	1	2	2	1
TRGP	Interval	3	2	2	5
	λ_0	0.01	5	5	5
	λ	0.0001	0.01	0.01	0.05
	Scale factor γ	1	1	2	1

Table 5. Hyperparameter settings for language tasks. We report only the hyperparameters used by PAPO and keep all other settings identical to [Liang et al. \(2026\)](#).

Setting	Hyperparameter	Order1	Order2
InfLoRA	Interval	1	5
	λ	0.5	0.5
GainLoRA	Interval	5	5
	λ	0.005	0.005

F. Additional Experiments

F.1. Singular Value spectra of ER, MAS, and GPM

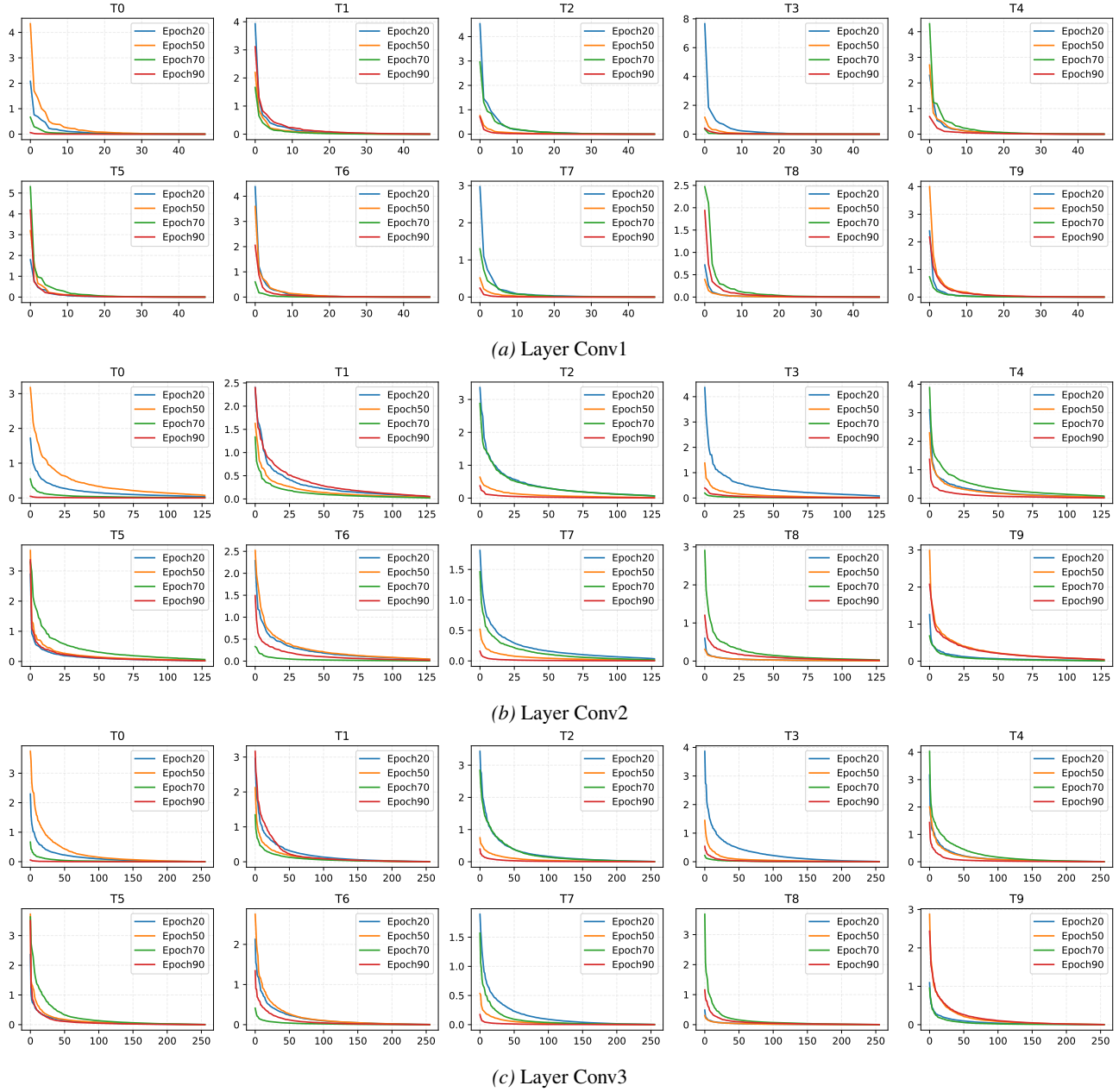


Figure 5. Singular value spectra for convolutional (Conv) layers Conv1, Conv2 and Conv3 at epochs 20, 50, 70 and 90 across tasks T0-T9 in ER. These results were obtained on the CIFAR-100 benchmark using an AlexNet backbone. Singular value collapse is also evident across multiple epochs.

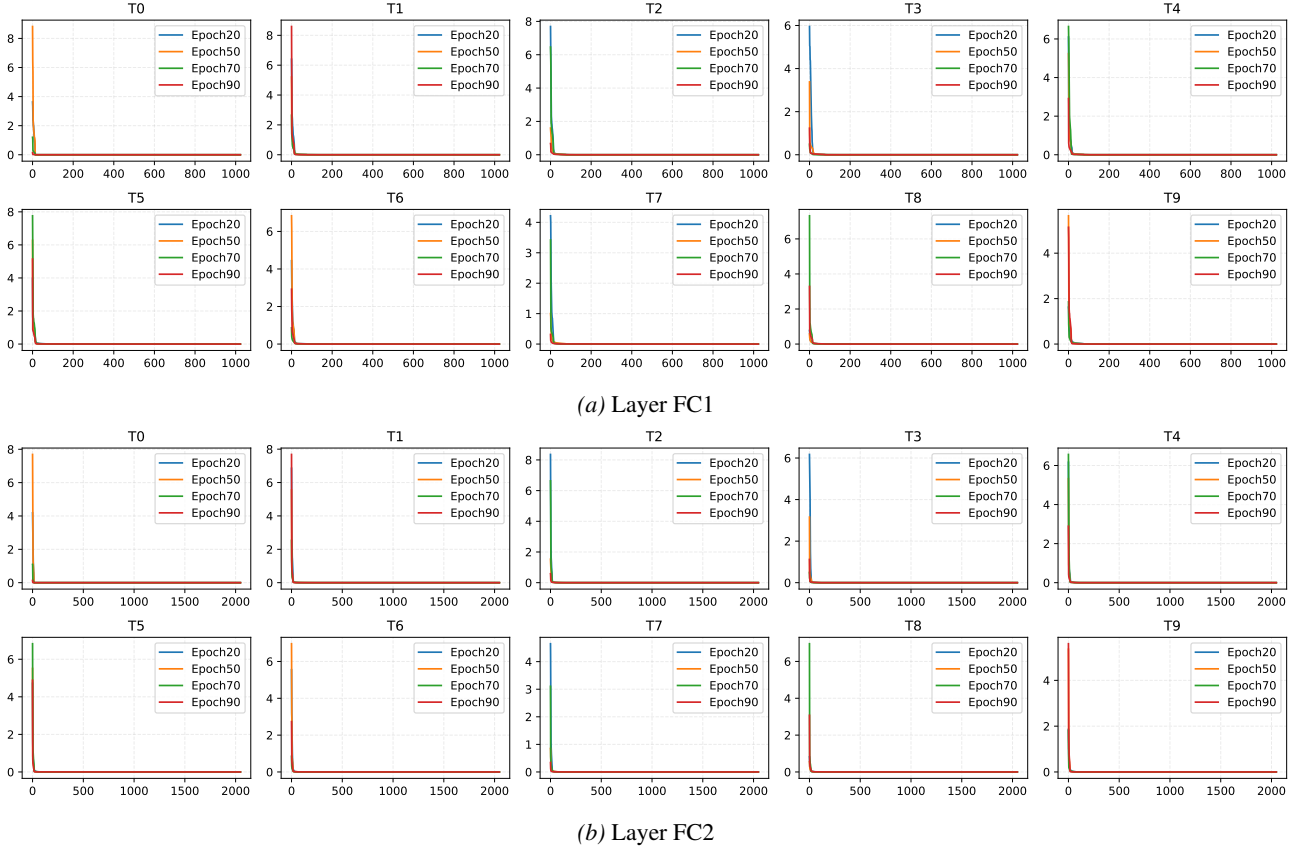


Figure 6. Singular value spectra for fully connected (FC) layers FC1 and FC2 at epochs 20, 50, 70 and 90 across tasks T0–T9 under **ER**. Results were obtained on the CIFAR-100 benchmark using an AlexNet backbone. The FC layers exhibit pronounced singular value collapse, characterized by an abrupt spectral drop and a long tail of near-zero singular values.

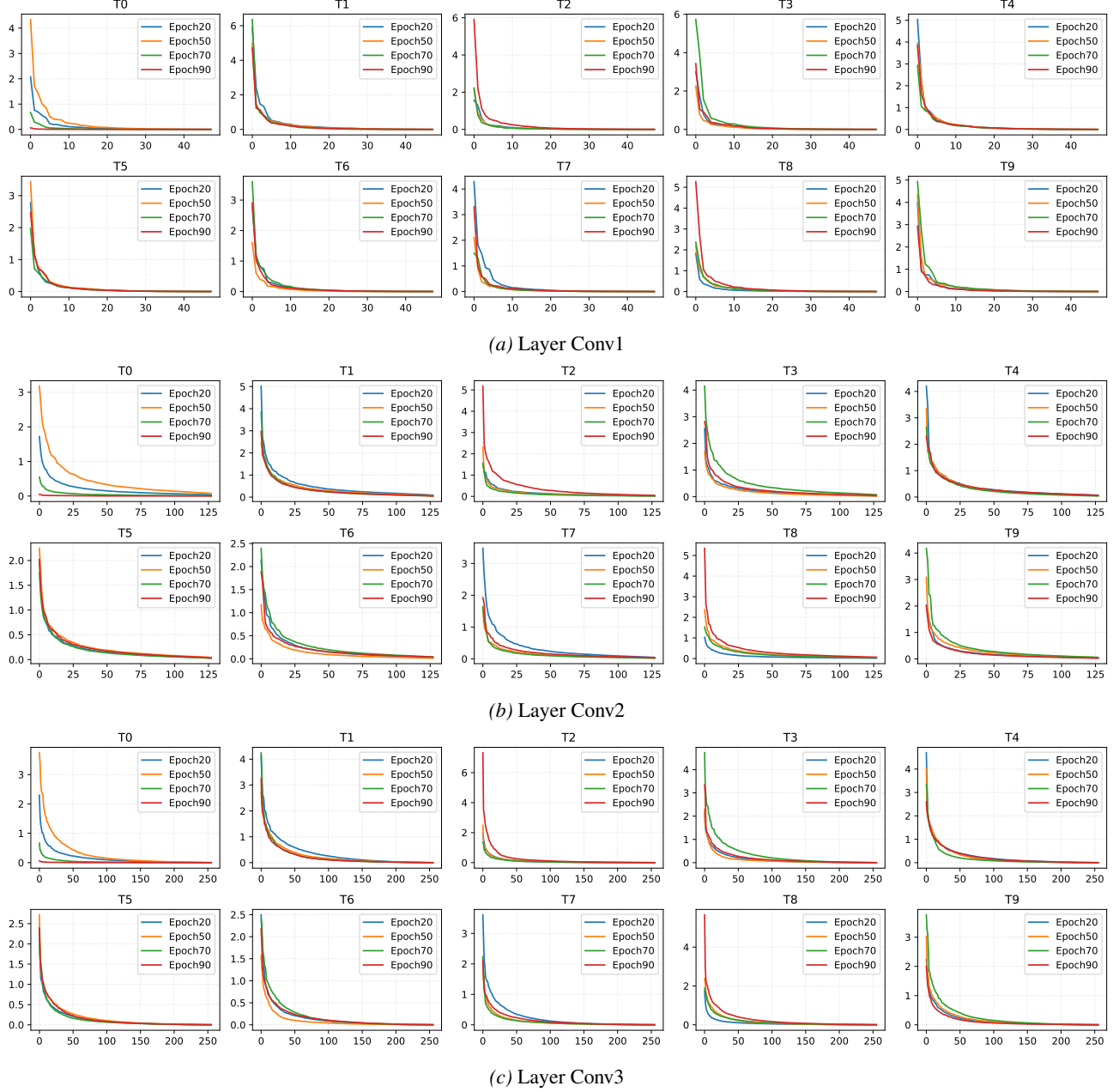


Figure 7. Singular value spectra for convolutional layers Conv1, Conv2 and Conv3 at epochs 20, 50, 70 and 90 across all tasks T0 to T9 in MAS. These results were obtained on the CIFAR-100 benchmark using an AlexNet backbone. Singular value collapse is also evident across multiple epochs.

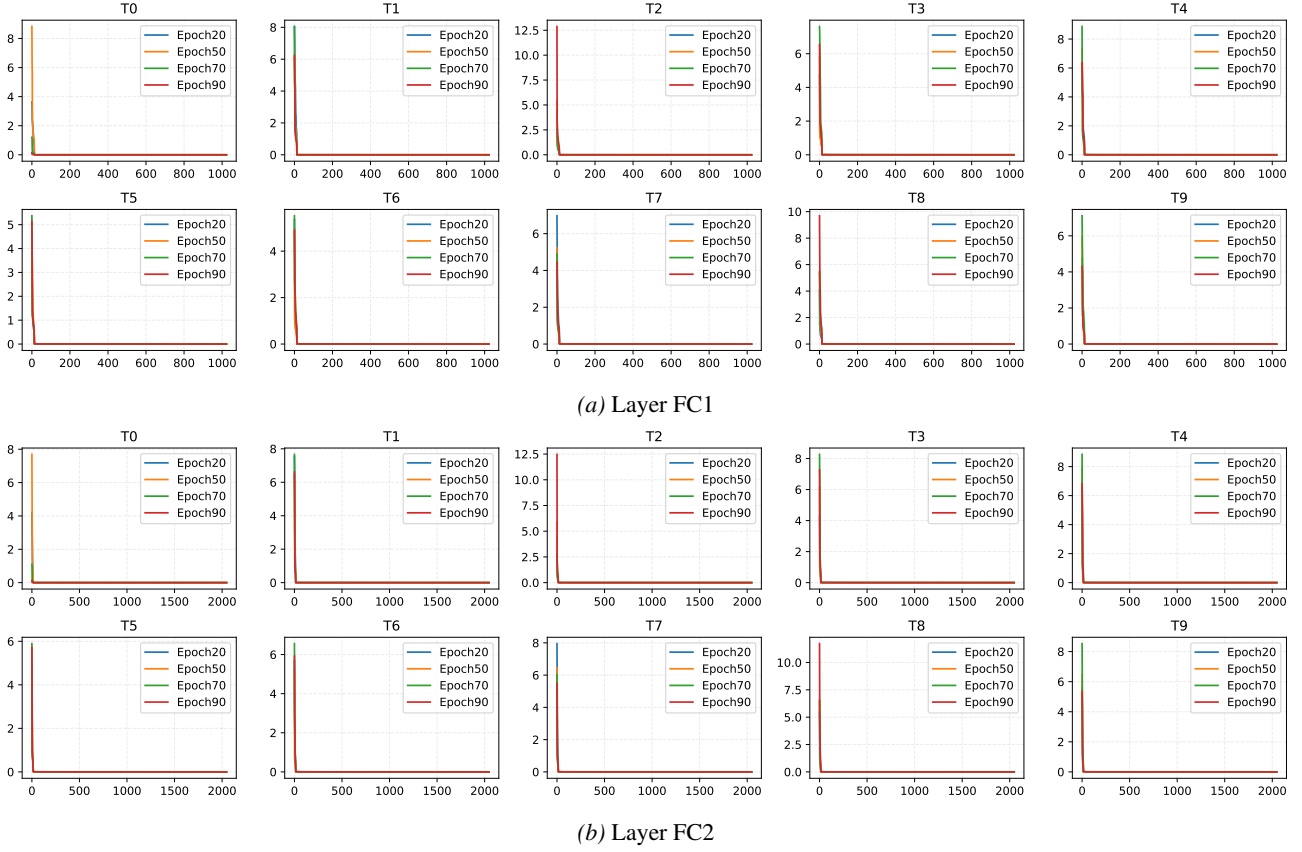


Figure 8. Singular value spectra for fully connected (FC) layers FC1 and FC2 at epochs 20, 50, 70 and 90 across tasks T0–T9 under MAS. Results were obtained on the CIFAR-100 benchmark using an AlexNet backbone. The FC layers exhibit pronounced singular value collapse, characterized by an abrupt spectral drop and a long tail of near-zero singular values.

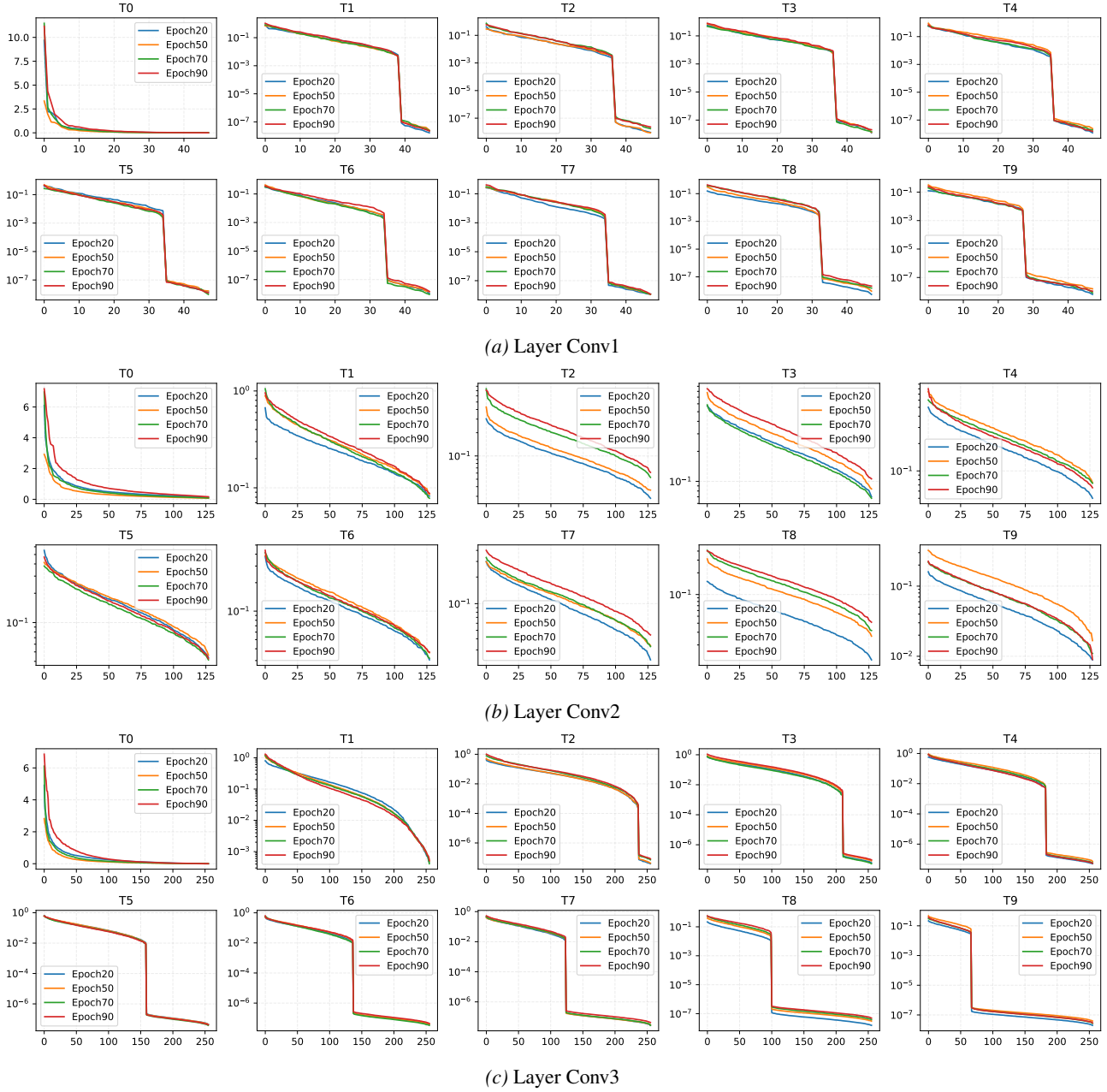


Figure 9. Singular value spectra for convolutional layers Conv1, Conv2 and Conv3 at epochs 20, 50, 70 and 90 across tasks T0–T9 under GPM. Results were obtained on the CIFAR-100 benchmark using an AlexNet backbone. To reveal the spectral cliff, the vertical axis is plotted on a logarithmic scale for tasks T1–T9. We observe a pronounced spectral cliff in Conv1 and Conv3. In Theorem 3.1 we show that this pattern arises when the intersection between the column space of the gradient and the subspace spanned by vectors from previous tasks is non-trivial. By contrast, Conv2 does not exhibit similar overlap with previous tasks.

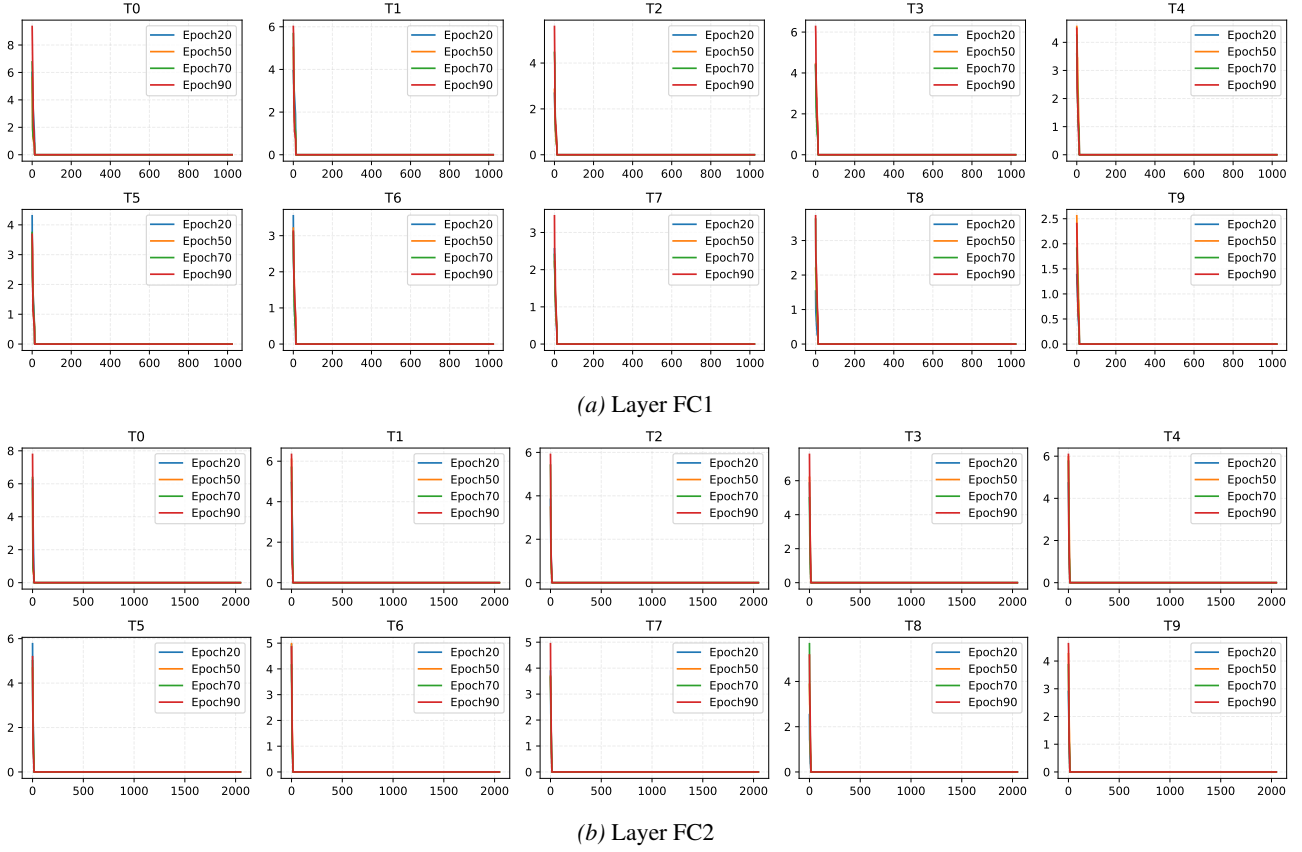


Figure 10. Singular value spectra for fully connected (FC) layers FC1 and FC2 at epochs 20, 50, 70 and 90 across tasks T0–T9 under **GPM**. Results were obtained on the CIFAR-100 benchmark using an AlexNet backbone. The FC layers exhibit pronounced singular value collapse, characterized by an abrupt spectral drop and a long tail of near-zero singular values.

F.2. Singular value spectra for different GPM thresholds

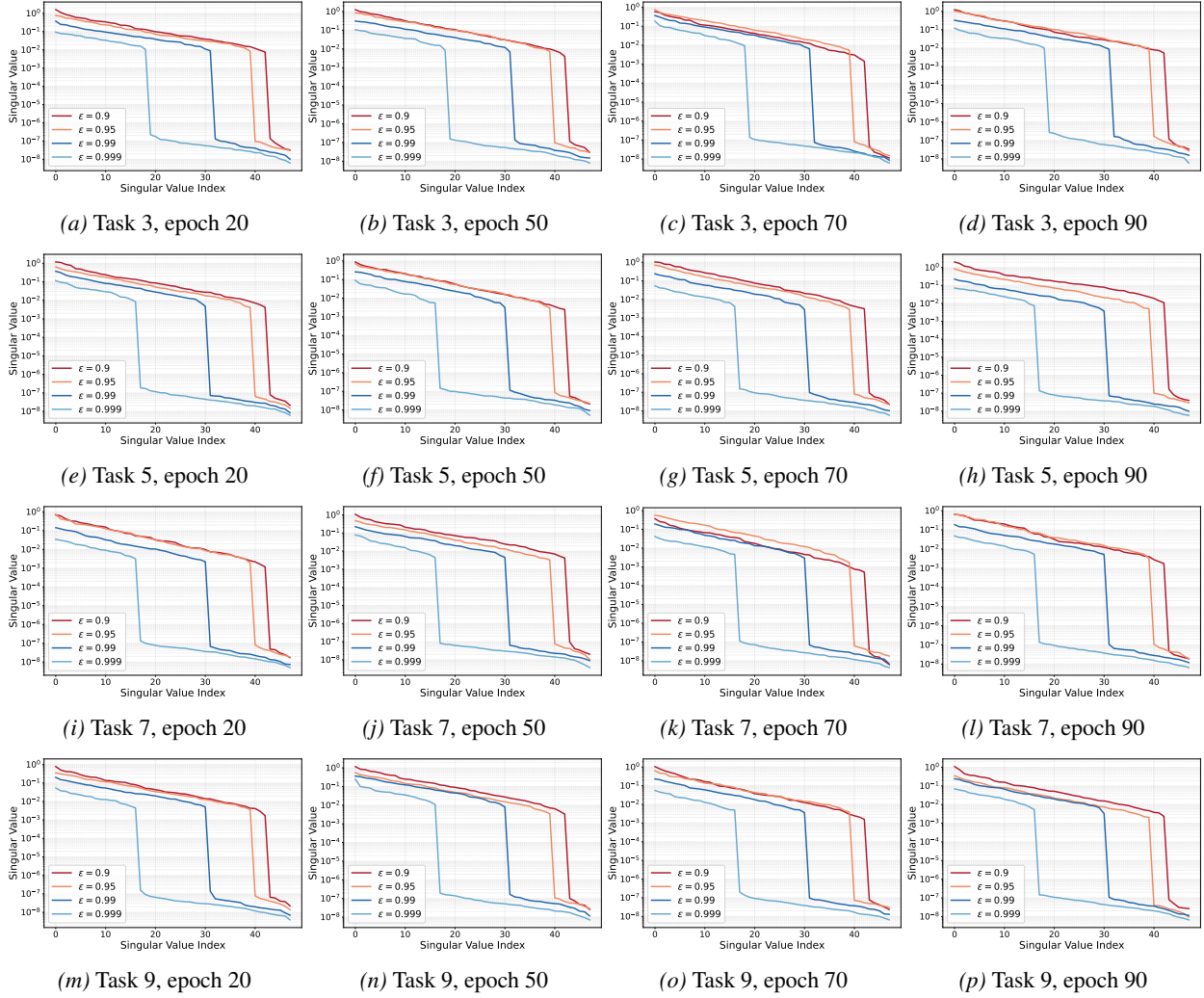


Figure 11. Conv1 singular value spectra under different GPM thresholds (i.e., 0.9, 0.95, 0.99 and 0.999). The horizontal axis denotes the singular value index, and the vertical axis shows the singular values on a logarithmic scale. For notational simplicity, we denote the projection threshold by ϵ . At any epoch (i.e., 20, 50, 70 and 90) and for any task (i.e., 3, 5, 7, 9), the spectral cliff shifts leftward as ϵ increases, indicating that larger projection thresholds induce more severe singular value collapse.

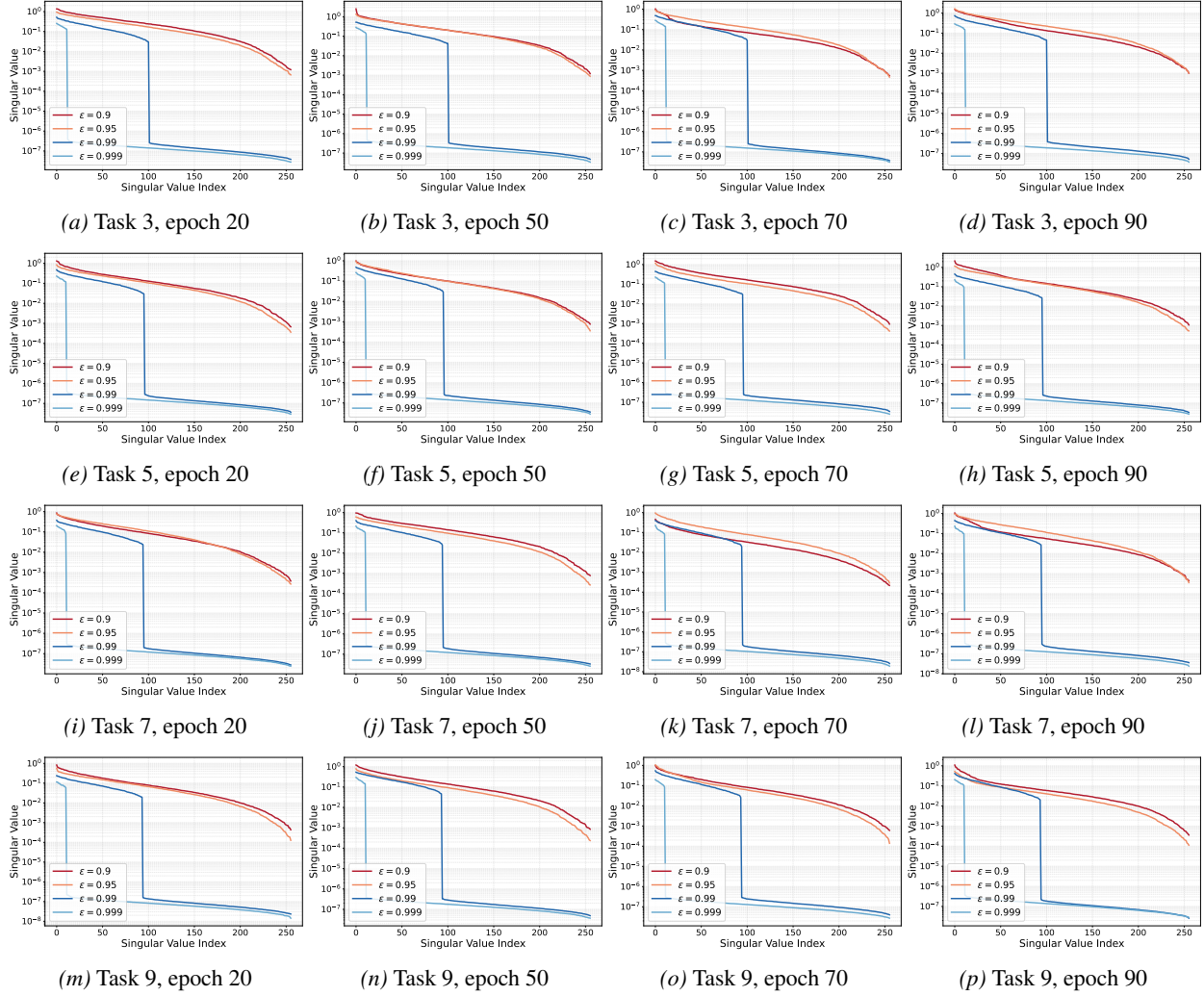


Figure 12. Conv3 singular value spectra under different GPM thresholds (i.e., 0.9, 0.95, 0.99 and 0.999). The horizontal axis denotes the singular value index, and the vertical axis shows the singular values on a logarithmic scale. At any epoch (i.e., 20, 50, 70 and 90) and for any task (i.e., 3, 5, 7, 9), the spectral cliff shifts leftward as ϵ increases, indicating that larger projection thresholds induce more severe singular value collapse.

F.3. Comparison of Singular Value Spectra With and Without PAPO

In this section, we compare the gradient singular value spectra produced by ER, MAS and GPM when run without PAPO and when augmented with PAPO to assess PAPO’s effect on singular-value collapse. We report results for epochs 20, 50, 70 and 90 for tasks T0, T3, T5, T7 and T9 in Figures 13 and 14. We observe that applying PAPO to these methods yields a clear recovery of singular values away from the near-zero regime. In particular, the bottom of the singular value cliff for GPM is lifted when PAPO is applied. Thus, incorporating PAPO mitigates singular value collapse.

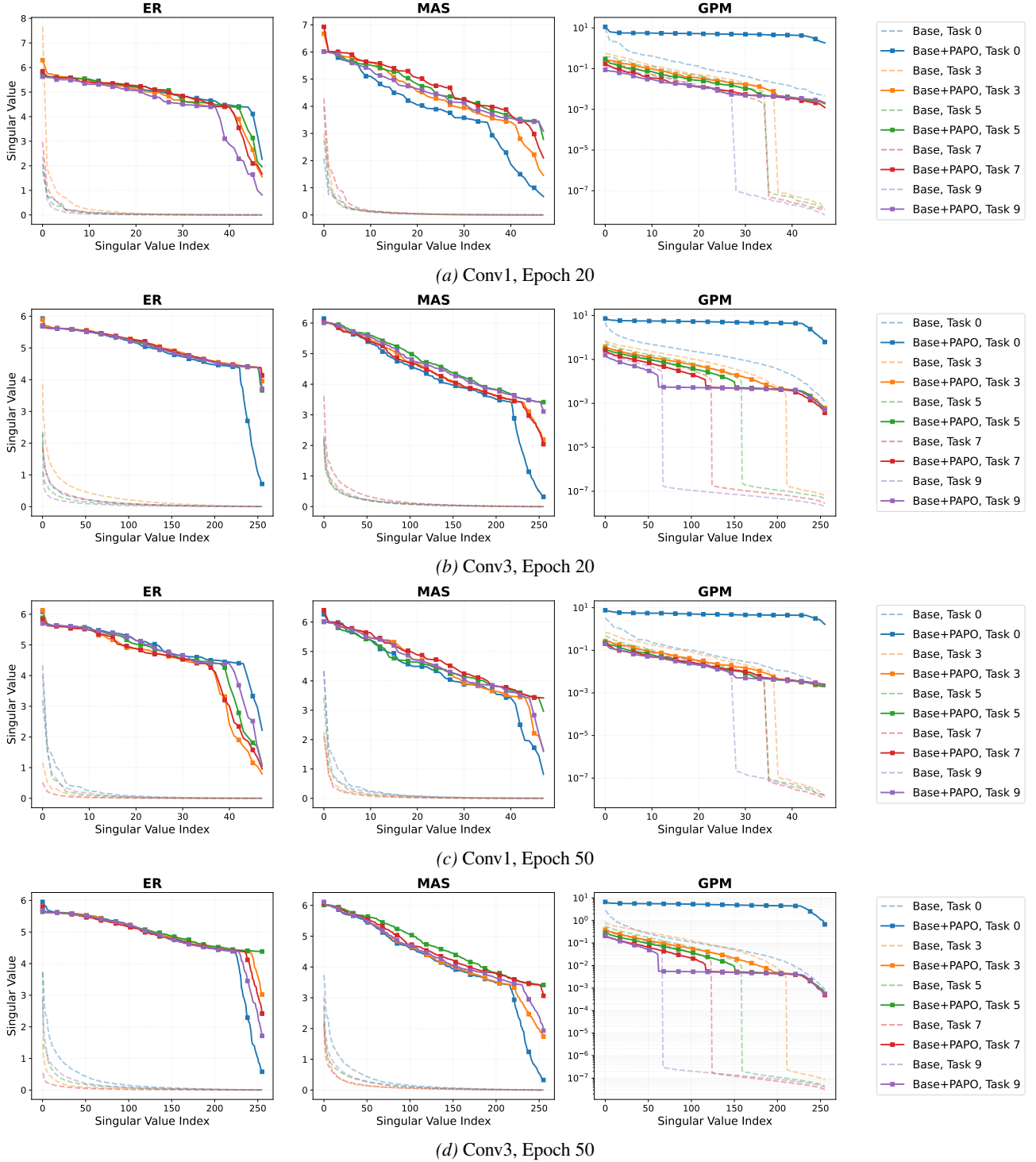


Figure 13. Comparison of gradient singular value spectra for methods with PAPO (solid lines) and without PAPO (dashed lines). Shown are Conv1 and Conv3 at epochs 20 and 50 for tasks T0, T3, T5, T7 and T9. In the legend, "Base" refers to the baselines without PAPO (i.e., ER, MAS, GPM).

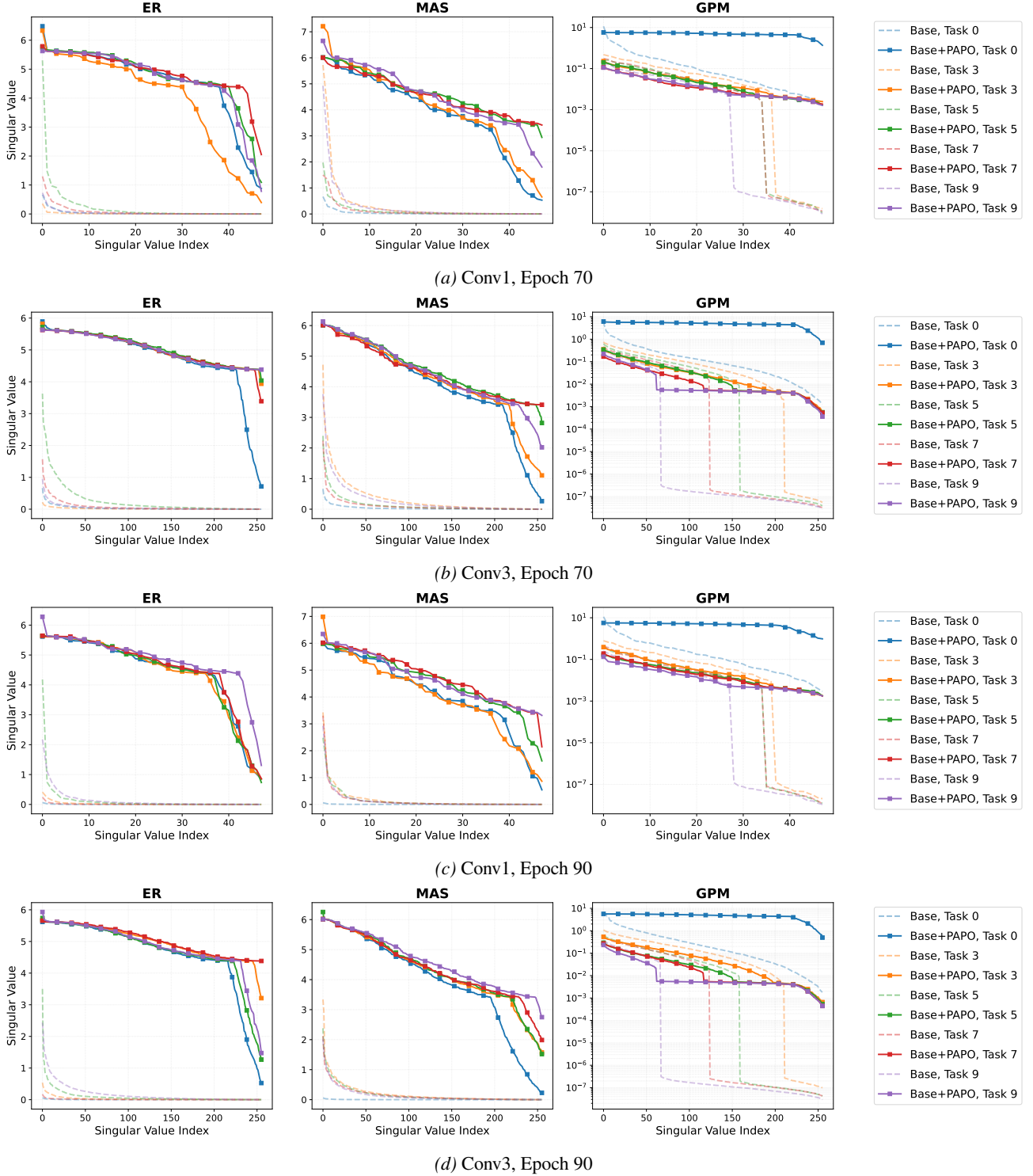


Figure 14. Comparison of gradient singular value spectra for methods with PAPO (solid lines) and without PAPO (dashed lines). Shown are Conv1 and Conv3 at epochs 70 and 90 for tasks T0, T3, T5, T7 and T9. In the legend, "Base" refers to the baselines without PAPO (i.e., ER, MAS, GPM).

F.4. Computational Costs Introduced by PAPO

To evaluate the computational efficiency of PAPO, we run experiments on task 1572 (Samsun summary) from the SuperNI benchmark using LLaMA-2-7B. Two factors primarily affect PAPO’s runtime: the PAPO application interval, which denotes the number of epochs between successive PAPO applications, and the number of Polar Express iterations, which denotes the iterations in Equation (6) used to approximate the polar factor of the gradient. As shown in Table 6, applying PAPO at every epoch increases runtime by 7.33% for InfLoRA and by 7.96% for GainLoRA relative to the baseline without PAPO. With a commonly used interval of 5, the runtime increases by 4.13% for InfLoRA and by 2.14% for GainLoRA. All experiments reported here were conducted on the same GPU and with identical hyperparameter settings. These results indicate that incorporating PAPO introduces only modest computational overhead in practice.

Table 6. Time cost for different PAPO application intervals (0, 1, 3, 5 and 10). The Polar Express approximation step is fixed at 5. An interval of 0 indicates that PAPO is not applied. Experiments were conducted on the same GPU and with identical hyperparameter settings.

Interval	0	1	3	5	10
InfLoRA	1622s	1741s	1705s	1689s	1632s
GainLoRA	1959s	2115s	2046s	2001s	1988s

Table 7. Time cost for different Polar Express iterations (i.e., 0, 1, 2, 3, 4, 5). The PAPO application interval is fixed at 5. An iteration of 0 indicates that PAPO is not applied. Experiments were conducted on the same GPU and with identical hyperparameter settings.

Iterations	0	1	2	3	4	5
InfLoRA	1622s	1663s	1675s	1681s	1686s	1689s
GainLoRA	1959s	1977s	1980s	1979s	1986s	2001s

F.5. Experiments in the Non-Continual Learning Setting

We evaluate PAPO in a non-continual learning setting by training ResNet-18 on CIFAR-100. As shown in Table 8, PAPO consistently reduces the error rate.

Table 8. Top-1 and top-5 errors in the non-CL setting for ResNet-18 on CIFAR-100.

Methods	Top1 error	Top5 error
SGD	24.39	6.95
+PAPO	23.77	6.89

F.6. Experiments on Online Continual Learning Benchmarks

We apply PAPO to online continual learning methods, including ER, OCM (Guo et al., 2022), and GSA (Guo et al., 2023), on the ImageNet100 benchmark. The hyperparameter settings of PAPO are listed in Table 9, and the other settings follow Wang et al. (2024a). The results are reported in Table 10. We find that PAPO also improves ACC and reduces FT in the online continual learning setting.

Table 9. Hyperparameter settings for Online CL Benchmarks.

Methods	λ	Interval	Scale factor
ER	0.01	10	1
OCM	1	1	1
GSA	0.5	5	1

Table 10. ACC and FT on the ImageNet100 under online continual learning setting. Higher ACC and lower FT indicate better generalization and less forgetting.

Methods	ACC↑	FT↓
ER	31.88	41.30
+PAPO	36.40	34.33
OCM	22.94	5.89
+PAPO	33.90	4.09
GSA	41.20	34.87
+PAPO	43.28	33.94

F.7. Other Experiments

In this section, we compare PAPO with several gradient correction-based methods, including isotropic perturbation, gradient rescaling, spectral regularization, SWA (Izmailov et al., 2018), and SAM (Foret et al., 2020).

For *isotropic perturbation*, we add isotropic Gaussian noise $\mathcal{G}$ to the gradient G as $G = G + \lambda \frac{\|\text{Polar}(G)\|_F}{\|\mathcal{G}\|_F + \epsilon} \mathcal{G}$, where the coefficient λ is searched over $\{0.0001, 0.001, 0.01, 0.1, 1\}$. For *gradient rescaling*, we normalize the gradient to unit Frobenius norm and then rescale it to match the norm of the gradient produced by PAPO, namely $G = \frac{\|\text{PAPO}(G)\|_F}{\|G\|_F + \epsilon} G$. We conduct these two experiments to show that the improvement brought by PAPO does not simply come from increasing the magnitude of the gradient.

We adopt *spectral regularization* from Lewandowski et al. (2025) and search over regularization coefficients in $\{0.0001, 0.001, 0.01, 0.1, 1\}$. For SWA, we use the implementation in `torch.optim.swa_utils` and search over both the starting epoch and the learning rate. We start SWA after 75% of the total epochs and search over learning rates in $\{0.001, 0.01, 0.1\}$. For SAM, we follow the SAM setting in Yang et al. (2023a).

Table 11. Comparison with additional methods. Higher ACC and BWT indicate better generalization and reduced forgetting.

Methods	ACC↑	BWT↑
ER	63.40 ± 2.94	−0.06 ± 0.01
+ Gaussian Noise	63.68 ± 2.26	−0.06 ± 0.01
+ Gradient Rescaling	63.59 ± 1.04	−0.05 ± 0.01
+ Spectral Regularization	64.62 ± 1.56	−0.05 ± 0.01
+ SWA	64.44 ± 0.56	−0.08 ± 0.00
+ SAM	65.10 ± 0.71	−0.04 ± 0.01
+ PAPO	72.41 ± 1.17	−0.03 ± 0.01
MAS	57.31 ± 2.64	−0.06 ± 0.02
+ Gaussian Noise	58.76 ± 1.64	−0.11 ± 0.01
+ Gradient Rescaling	60.75 ± 0.99	−0.03 ± 0.01
+ Spectral Regularization	56.53 ± 0.78	−0.13 ± 0.01
+ SWA	61.67 ± 2.95	−0.06 ± 0.02
+ SAM	62.94 ± 1.07	−0.08 ± 0.01
+ PAPO	62.07 ± 1.71	−0.02 ± 0.01
GPM	60.20 ± 1.98	−0.04 ± 0.02
+ Gaussian Noise	62.38 ± 3.63	−0.02 ± 0.02
+ Gradient Rescaling	61.11 ± 1.46	−0.02 ± 0.01
+ Spectral Regularization	62.53 ± 2.39	−0.02 ± 0.01
+ SWA	62.68 ± 1.52	−0.03 ± 0.01
+ SAM	67.74 ± 1.39	−0.01 ± 0.00
+ PAPO	69.10 ± 1.48	+0.00 ± 0.00